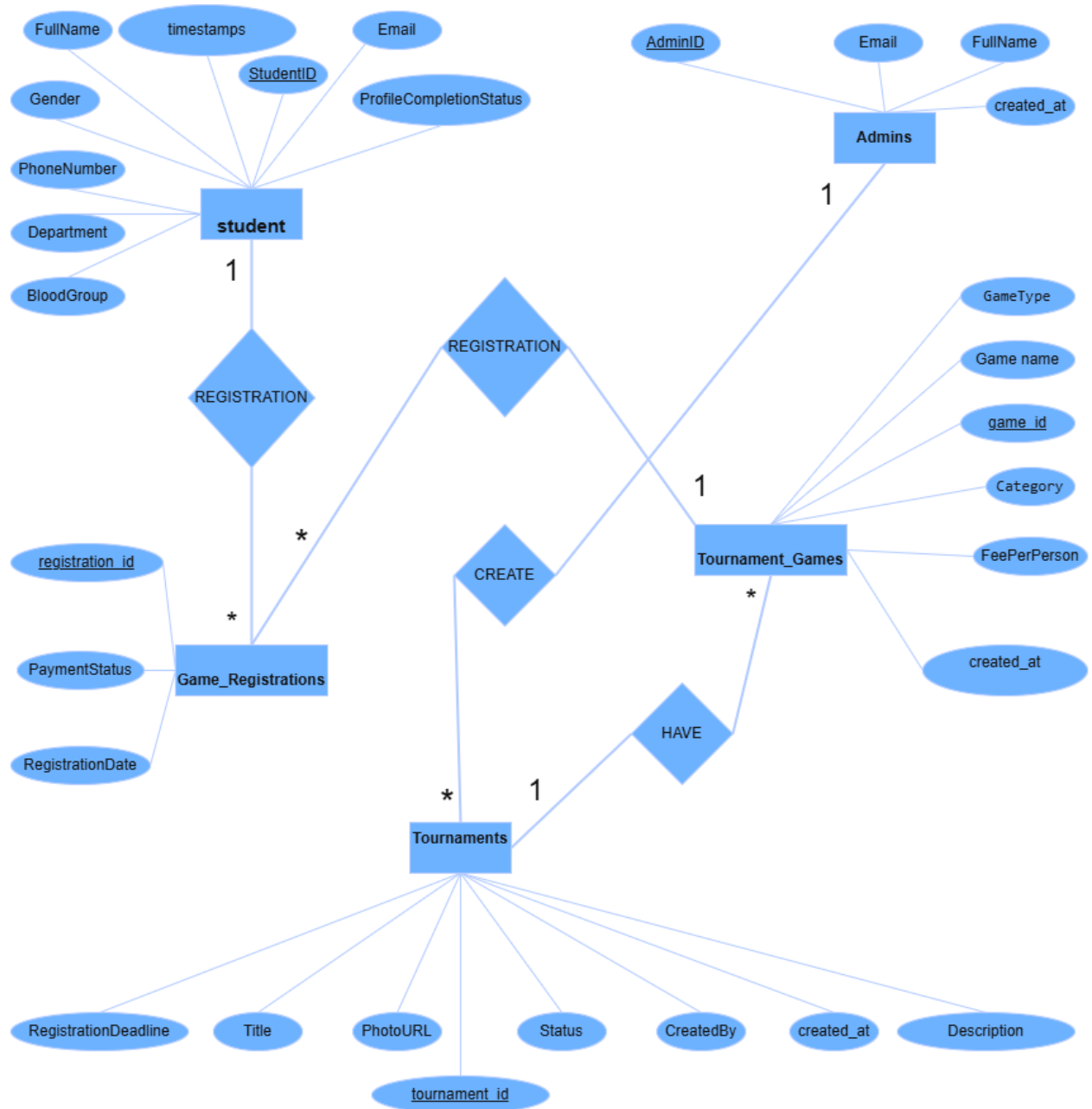


Contents

Project Updates	3
ER Diagram	3
Normalization	4
Basic PL/SQL	9
Advance PL/SQL.....	23
Query Writing.....	40
Relational Algebra	64
User Interface and Code Screenshots.....	65
.....	

Project Updates

ER Diagram



Normalization

Registration

UNF:

Registration (student_id, email, full_name, gender, program_level, department, blood_group, registration_id, registration_date, payment_status, registration_id, registration_date, payment_status)

1NF:

There is no multivalued attribute. Relation already in 1NF.

1. student_id, email, full_name, gender, program_level, department, blood_group, registration_id, registration_date, payment_status, registration_id, registration_date, payment_status.

2NF:

1. student_id, email, full_name, gender, program_level, department, blood_group)
2. registration_id, registration_date, payment_status.

3NF:

No transitive dependency Relation already in 3NF.

1. student_id, email, full_name, gender, program_level, department, blood_group
2. registration_id, registration_date, payment_status

Table Creation:

1. student_id, email, full_name, gender, program_level, department, blood_group
2. registration_id, registration_date, payment_status, student_id

Create

UNF:

Creation(admin_id, admin_name, admin_email, created_at, tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by)

1NF:

There is no multivalued attribute. Relation already in 1NF.

1. admin_id, admin_name, admin_email, created_at, tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by.

2NF:

1. admin_id, admin_name, admin_email, created_at

2. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by

3NF:

No transitive dependency Relation already in 3NF.

1. admin_id, admin_name, admin_email, created_at

2. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by

Table Creation:

1. admin_id, admin_name, admin_email, created_at

2. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by,
admin_id

Have

UNF:

Have(tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by, game_id, category, game_name, game_type, fee_per_person, created_at)

1NF:

There is no multivalued attribute. All attributes are atomic. Relation already in 1NF

1. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by, game_id, category, game_name, game_type, fee_per_person, created_at

2NF:

1. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by

2. game_id, category, game_name, game_type, fee_per_person, created_at

3NF:

No transitive dependency Relation already in 3NF.

1. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by

2. game_id, category, game_name, game_type, fee_per_person, created_at

Table Creation:

1. tournament_id, title, photo_url, registration_deadline, status, description, created_at, create_by

2. game_id, category, game_name, game_type, fee_per_person, created_at, **tournament_id**

Registration (Game_Registrations ↔ Tournament_Games)

UNF:

Registration (registration_id, PaymentStatus, registrationdate, game_id, category, game_name, game_type, fee_per_person, created_at)

1NF:

There is no multivalued attribute. All attributes are atomic. Relation already in 1NF.

1. registration_id, PaymentStatus, registrationdate, game_id, category, game_name, game_type, fee_perperson, created_at

2NF:

1. registration_id, PaymentStatus, registrationdate

2. game_id, category, game_name, game_type, fee_per_person, created_at

3NF:

No transitive dependency Relation already in 3NF.

1. registration_id, PaymentStatus, registrationdate

2. game_id, category, game_name, game_type, fee_per_person, created_at

Table Creation:

1. registration_id, PaymentStatus, registrationdate

2. game_id, category, game_name, game_type, fee_per_person, created_at, **registration_id**

Temporary Tables:

1. student_id, email, full_name, gender, program_level, department, blood_group
2. ~~registration_id~~, registration_date, payment_status, **student_id**
3. admin_id, admin_name, admin_email, created_at
4. ~~tournament_id~~, title, photo_url, registration_deadline, status, description, created_at, CreatedBy, **admin_id**
5. tournament_id, title, photo_url, registration_deadline, status, description, created_at, CreatedBy
6. ~~game_id~~, category, game_name, game_type, fee_per_person, created_at, **tournament_id**
7. registration_id, registration_date, payment_status
8. game_id, category, game_name, game_type, fee_per_person, created_at, **registration_id**

Final Tables:

1. student_id, email, full_name, gender, program_level, department, blood_group
2. admin_id, admin_name, admin_email, created_at
3. tournament_id, title, photo_url, registration_deadline, status, description, created_at, CreatedBy
4. registration_id, registration_date, payment_status,
5. game_id, category, game_name, game_type, fee_per_person, created_at, **registration_id**

Basic PL/SQL

2 Variables

Question 1 :

Write a PL/SQL block that declares a variable to store the total number of users, retrieves the count of records from the users table, and displays the result using DBMS_OUTPUT.PUT_LINE?

Answer :

```
DECLARE
    v_total_users NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total_users FROM users;
    DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_total_users);
END;
```

The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes 'ORACLE Database Express Edition', 'User AUB PORTAL', and links for 'Home', 'Logout', and 'Help'. The breadcrumb trail is 'Home > SQL > SQL Commands'. The main editor area has a toolbar with 'Autocommit' (checked), 'Display' (set to 10), 'Save', and 'Run'. The SQL code entered is:
DECLARE
 v_total_users NUMBER;
BEGIN
 SELECT COUNT(*) INTO v_total_users FROM users;
 DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_total_users);
END;
/
The 'Run' button has been clicked, and the results pane shows:
Total Users: 5
Statement processed.
0.00 seconds
The footer of the interface displays 'Language: en-us' on the left and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.' on the right.



Question 2 :

Write a PL/SQL block that declares a variable to store a user's full name, retrieves the full_name of the user whose id is 1 from the users table, and displays it using DBMS_OUTPUT.PUT_LINE.?

Answer :

DECLARE

 v_first_user_name VARCHAR2(100);

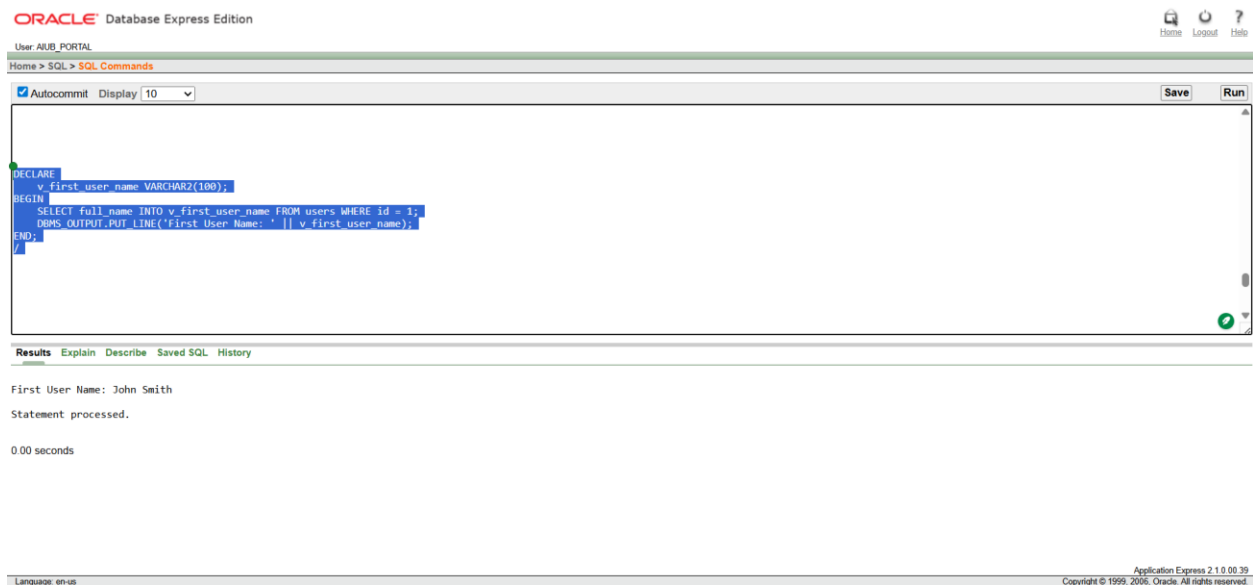
BEGIN

 SELECT full_name INTO v_first_user_name FROM users WHERE id = 1;

 DBMS_OUTPUT.PUT_LINE('First User Name: ' || v_first_user_name);

END;

/



2 Operators

Question 1 :

Write a PL/SQL block that declares a variable, calculates the sum of 5 and 10 using an arithmetic operator, and displays the result using DBMS_OUTPUT.PUT_LINE?

Answer :

DECLARE

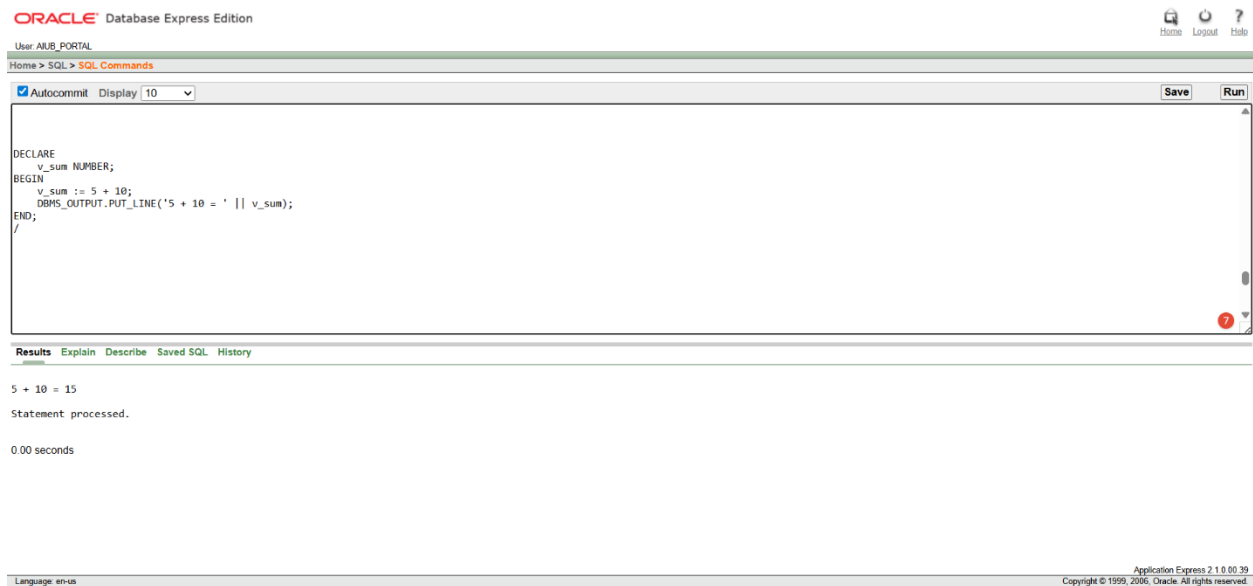
 v_sum NUMBER;

BEGIN

 v_sum := 5 + 10;

 DBMS_OUTPUT.PUT_LINE('5 + 10 = ' || v_sum);

END;



Question 2 :

Write a PL/SQL block that counts the number of records in the users table, and then:

- If the count is greater than 3, display "More than 3 users."
- Otherwise, display "3 or fewer users."

Answer :

DECLARE

 v_count NUMBER;

BEGIN

 SELECT COUNT(*) INTO v_count FROM users;

 IF v_count > 3 THEN

 DBMS_OUTPUT.PUT_LINE('More than 3 users.');

 ELSE

 DBMS_OUTPUT.PUT_LINE('3 or fewer users.');

 END IF;

END;

The screenshot displays the Oracle Database Express Edition interface. The top bar shows the user as 'AIRB_PORTAL'. The main window is titled 'SQL Commands' and contains the following PL/SQL code:

```
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM users;
  IF v_count > 3 THEN
    DBMS_OUTPUT.PUT_LINE('More than 3 users.');
```

The code is executed, and the results are shown in the 'Results' tab below the editor. The output is:

```
More than 3 users.
Statement processed.
0.00 seconds
```

The bottom status bar indicates the application version as 'Application Express 2.1.0.00.39' and the copyright as 'Copyright © 1999, 2006, Oracle. All rights reserved.'

2 Single-row function

Question 1 :

Write a PL/SQL block that declares a variable, retrieves the full_name of the user with id = 1 from the users table, converts it to uppercase using a single-row function, and displays the result?

Answer :

DECLARE

 v_upper_name VARCHAR2(100);

BEGIN

 SELECT UPPER(full_name) INTO v_upper_name FROM users WHERE id = 1;

 DBMS_OUTPUT.PUT_LINE('Upper case name: ' || v_upper_name);

END;

The screenshot shows the Oracle Database Express Edition web interface. At the top, it says "ORACLE Database Express Edition" and "User: AUB PORTAL". Below the navigation bar, there's a "SQL Commands" section. The main area contains a text editor with the following PL/SQL code:

```
DECLARE
  v_upper_name VARCHAR2(100);
BEGIN
  SELECT UPPER(full_name) INTO v_upper_name FROM users WHERE id = 1;
  DBMS_OUTPUT.PUT_LINE('Upper case name: ' || v_upper_name);
END;
```

Below the code editor, the "Results" tab is selected, showing the output: "Upper case name: JOHN SMITH". Below that, it says "Statement processed." and "0.00 seconds". At the bottom, there's a footer with "Language: en-us" and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved."



Question 2 :

Write a PL/SQL block that declares a variable, retrieves the full_name of the user with id = 1 from the users table, calculates the length of the name using a single-row function, and displays?

Answer :

DECLARE

 v_name_length NUMBER;

BEGIN

 SELECT LENGTH(full_name) INTO v_name_length FROM users WHERE id = 1;

 DBMS_OUTPUT.PUT_LINE('Name length: ' || v_name_length);

END;

The screenshot displays the Oracle Database Express Edition interface. At the top, the title bar reads "ORACLE Database Express Edition". Below it, the user is identified as "User: A1UB_PORTAL". The main window shows the "SQL Commands" tab with a text area containing the following PL/SQL code:

```
DECLARE
    v_name_length NUMBER;
BEGIN
    SELECT LENGTH(full_name) INTO v_name_length FROM users WHERE id = 1;
    DBMS_OUTPUT.PUT_LINE('Name length: ' || v_name_length);
END;
```

Below the code editor, the "Results" tab is active, showing the output of the execution:

```
Name length: 10
Statement processed.
0.02 seconds
```

The bottom status bar indicates the language is "en-us" and the application version is "Application Express 2.1.0.00.39".

2 group function

Question 1 :

Write a PL/SQL block that declares a variable to store the total number of users, retrieves the count of records from the users table, and displays the total?

Answer :

DECLARE

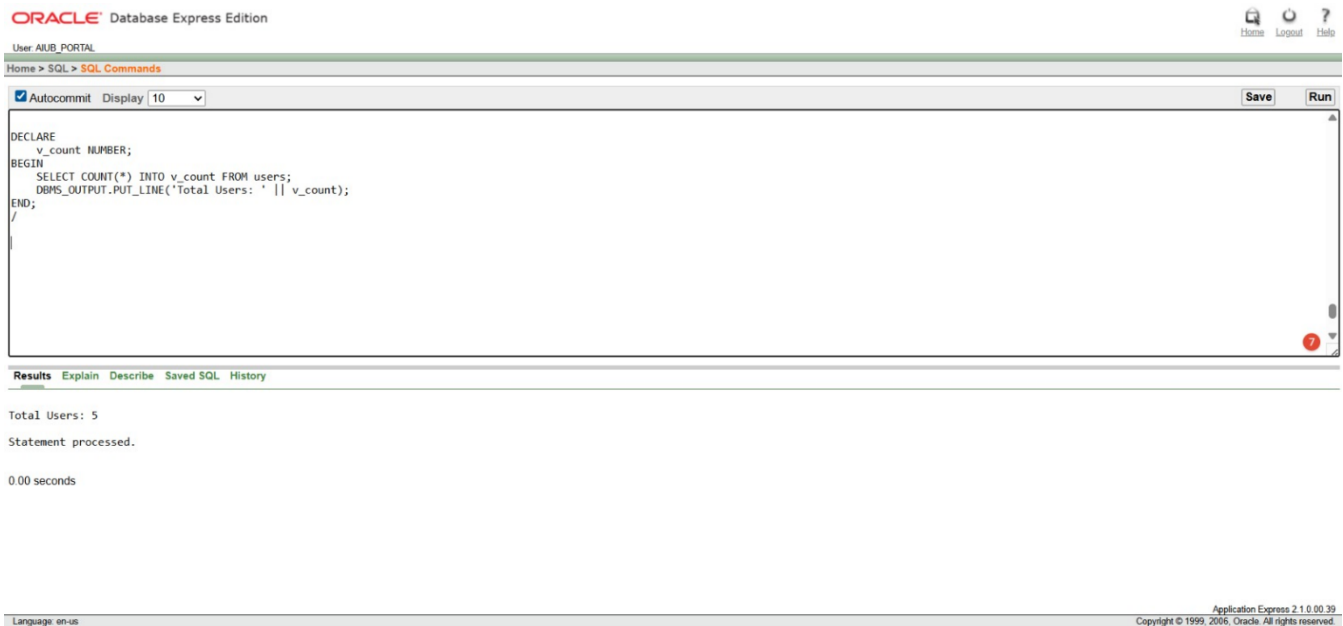
 v_count NUMBER;

BEGIN

 SELECT COUNT(*) INTO v_count FROM users;

 DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_count);

END;



Question 2 :

Write a PL/SQL block to **calculate the average length of the full_name field** in the users table and display the result rounded to 2 decimal places?

Answer :

DECLARE

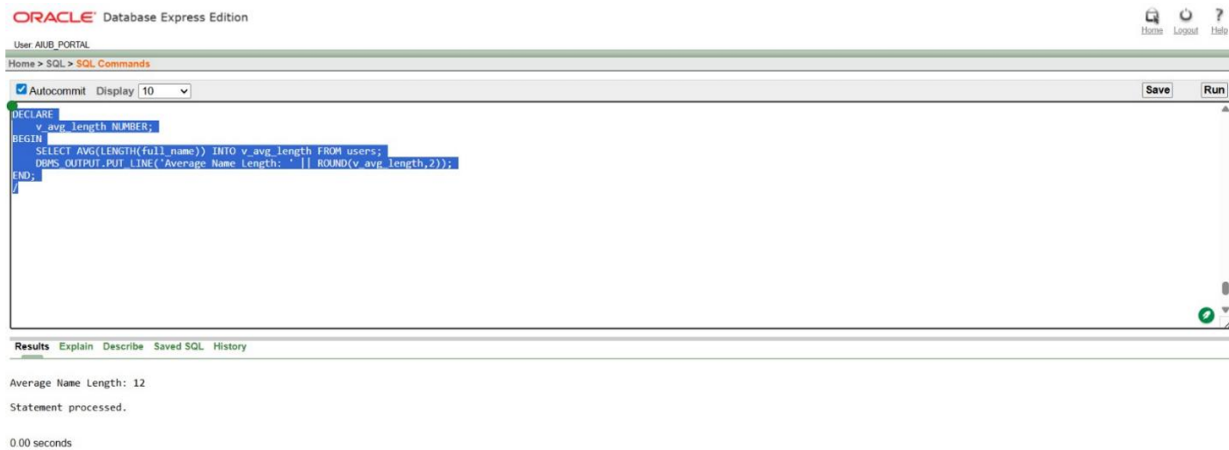
 v_avg_length NUMBER;

BEGIN

 SELECT AVG(LENGTH(full_name)) INTO v_avg_length FROM users;

 DBMS_OUTPUT.PUT_LINE('Average Name Length: ' || ROUND(v_avg_length, 2));

END;



The screenshot shows the Oracle Database Express Edition interface. The SQL Command window contains the following PL/SQL block:

```
DECLARE
    v_avg_length NUMBER;
BEGIN
    SELECT AVG(LENGTH(full_name)) INTO v_avg_length FROM users;
    DBMS_OUTPUT.PUT_LINE('Average Name Length: ' || ROUND(v_avg_length, 2));
END;
```

The block is executed successfully, and the output is displayed in the Results window:

```
Average Name Length: 12
Statement processed.
0.00 seconds
```

2 conditional statements

Question 1 :

Write a PL/SQL block that counts the number of records in the users table and uses a conditional statement (IF...ELSE) to display:

- "More than 3 users." if the count is greater than 3
- "3 or fewer users." otherwise.

Answer :

DECLARE

 v_count NUMBER;

BEGIN

 SELECT COUNT(*) INTO v_count FROM users;

 IF v_count > 3 THEN

 DBMS_OUTPUT.PUT_LINE('More than 3 users.');

 ELSE

 DBMS_OUTPUT.PUT_LINE('3 or fewer users.');

 END IF; END;

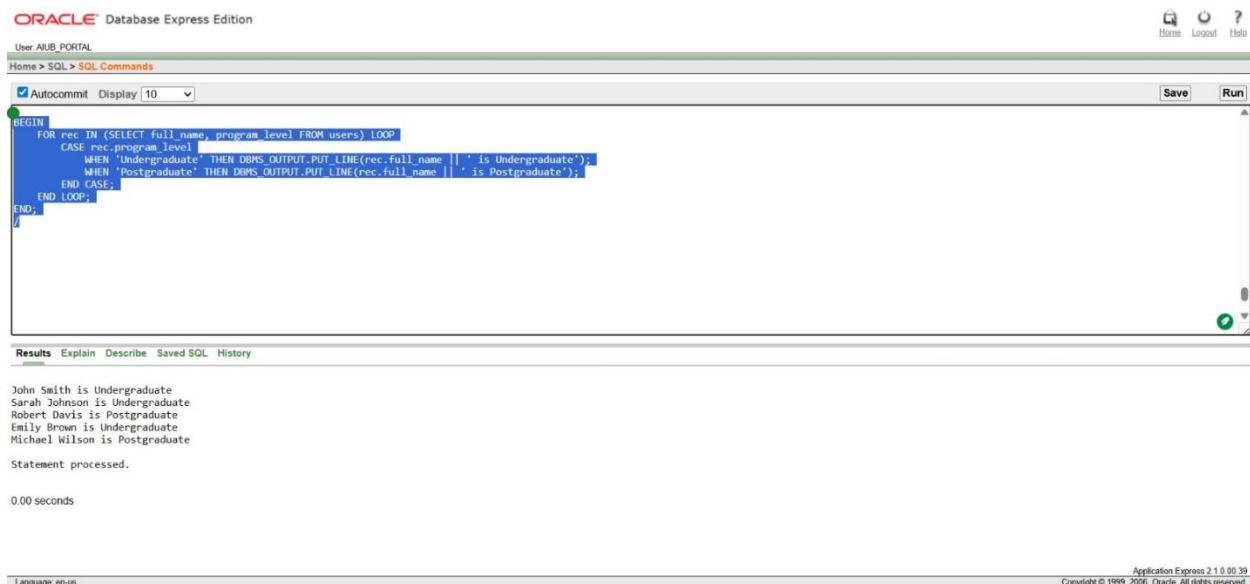


Question 2 :

Write a PL/SQL block that loops through all users in the users table and, using a CASE statement, displays each user's full_name along with whether they are "Undergraduate" or "Postgraduate" based on their program_level?

Answer :

```
BEGIN
  FOR rec IN (SELECT full_name, program_level FROM users) LOOP
    CASE rec.program_level
      WHEN 'Undergraduate' THEN DBMS_OUTPUT.PUT_LINE(rec.full_name || ' is Undergraduate');
      WHEN 'Postgraduate' THEN DBMS_OUTPUT.PUT_LINE(rec.full_name || ' is Postgraduate');
    END CASE;
  END LOOP;
END;
```



2 Subquery

Question 1 :

Write a PL/SQL block that declares a variable, counts the number of users who belong to the same department as the user with id = 1 using a subquery, and displays the result?

Answer :

DECLARE

 v_dept_count NUMBER;

BEGIN

 SELECT COUNT(*) INTO v_dept_count

 FROM users

 WHERE department = (SELECT department FROM users WHERE id = 1);

 DBMS_OUTPUT.PUT_LINE('Users in same department as user 1: ' || v_dept_count);

END;

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes 'ORACLE Database Express Edition' and links for 'Home', 'Logout', and 'Help'. The breadcrumb trail shows 'Home > SQL > SQL Commands'. The main editor area contains a PL/SQL block with the following code:

```
BEGIN
  FOR rec IN (
    SELECT u.full_name AS user_name, a.full_name AS admin_name
    FROM users u
    JOIN admins a ON a.id = 1
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);
  END LOOP;
END;
```

Below the editor, the 'Results' tab is active, showing the output of the execution:

```
User: John Smith, Admin: Admin One
User: Sarah Johnson, Admin: Admin One
User: Robert Davis, Admin: Admin One
User: Emily Brown, Admin: Admin One
User: Michael Wilson, Admin: Admin One
```

The status bar at the bottom indicates 'Statement processed.' and '0.02 seconds'. The footer shows 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

Question 2 :

Write a PL/SQL block that declares a variable, retrieves the email of the user with the highest id from the users table using a subquery, and displays?

Answer :

DECLARE

 v_email VARCHAR2(100);

BEGIN

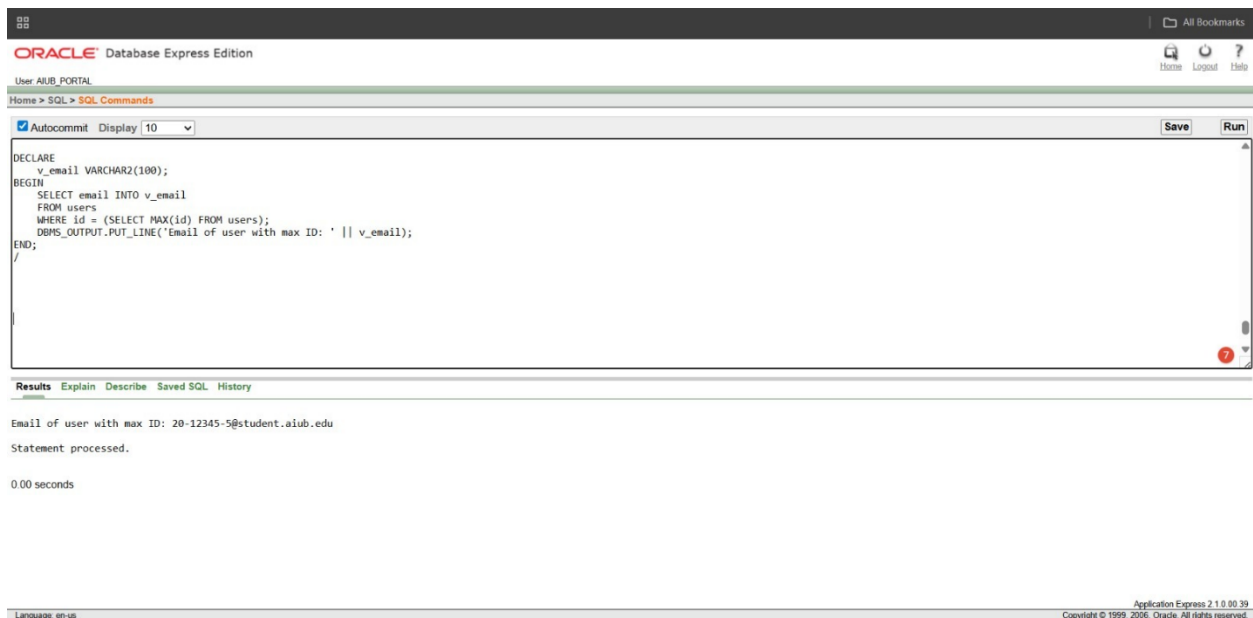
 SELECT email INTO v_email

 FROM users

 WHERE id = (SELECT MAX(id) FROM users);

 DBMS_OUTPUT.PUT_LINE('Email of user with max ID: ' || v_email);

END;



2 Joining

Question 1 :

Write a PL/SQL block that uses a JOIN to retrieve each user's full_name along with the full_name of the admin with id = 1, and displays the result?

Answer :

BEGIN

FOR rec IN (

SELECT u.full_name AS user_name, a.full_name AS admin_name

FROM users u

JOIN admins a ON a.id = 1) LOOP

DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);

END LOOP;

The screenshot displays the Oracle Database Express Edition web interface. At the top, it shows 'ORACLE Database Express Edition' and navigation links for 'Home', 'Logout', and 'Help'. Below this, the user is identified as 'User: AUB_PORTAL'. The main area is titled 'SQL Commands' and contains a text editor with the following PL/SQL code:

```
BEGIN
  FOR rec IN (
    SELECT u.full_name AS user_name, a.full_name AS admin_name
    FROM users u
    JOIN admins a ON a.id = 1
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);
  END LOOP;
END;
```

Buttons for 'Autocommit', 'Display 10', 'Save', and 'Run' are visible above the editor. Below the editor, there are tabs for 'Results', 'Explain', 'Describe', 'Saved SQL', and 'History'. The 'Results' tab is active, showing the output of the query:

```
User: John Smith, Admin: Admin One
User: Sarah Johnson, Admin: Admin One
User: Robert Davis, Admin: Admin One
User: Emily Brown, Admin: Admin One
User: Michael Wilson, Admin: Admin One
```

Below the output, it states 'Statement processed.' and '0.02 seconds'. At the bottom of the interface, it shows 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

Question 2 :

Write a PL/SQL block that retrieves the full_name of users with id less than or equal to 3 and pairs them with the admin whose id = 2 using an implicit join, then displays each pair in the format: User: <user_name>, Admin: <admin_name>?

Answer :

BEGIN

FOR rec IN (

SELECT u.full_name AS user_name, a.full_name AS admin_name

FROM users u, admins a

WHERE a.id = 2 AND u.id <= 3

) LOOP

DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);

END LOOP;

END;

The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes 'ORACLE Database Express Edition', 'User AUB_PORTAL', and links for 'Home', 'Logout', and 'Help'. The breadcrumb trail is 'Home > SQL > SQL Commands'. The main area contains a text editor with the following PL/SQL code:

```
BEGIN
  FOR rec IN (
    SELECT u.full_name AS user_name, a.full_name AS admin_name
    FROM users u, admins a
    WHERE a.id = 2 AND u.id <= 3
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);
  END LOOP;
END;
```

Below the editor, the 'Results' tab is active, displaying the output of the query:

```
User: John Smith, Admin: Admin Two
User: Sarah Johnson, Admin: Admin Two
User: Robert Davis, Admin: Admin Two
```

Below the output, it says 'Statement processed.' and '0.00 seconds'. The footer of the interface shows 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

Advance PL/SQL

2 Stored functions

Question 1:

Write a PL/SQL function named `get_total_users` that returns the total number of records from the `users` table. Also write an anonymous PL/SQL block to call this function and display the result using `DBMS_OUTPUT` ?

Answer :

```
CREATE OR REPLACE FUNCTION get_total_users RETURN NUMBER IS
```

```
    v_count NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO v_count FROM users;
```

```
    RETURN v_count;
```

```
END;
```

```
/
```

Test Function 1

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Total Users: ' || get_total_users);
```

```
END;
```

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes the Oracle logo, the text "Database Express Edition", and user information "User: AUB_PORTAL". On the right, there are links for "Home", "Logout", and "Help". Below the navigation bar, the breadcrumb "Home > SQL > SQL Commands" is visible. The main area contains a text editor with the following PL/SQL code:

```
CREATE OR REPLACE FUNCTION get_total_users RETURN NUMBER IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM users;
    RETURN v_count;
END;
/
BEGIN
    DBMS_OUTPUT.PUT_LINE('Total Users: ' || get_total_users);
END;
/
```

Buttons for "Autocommit", "Display 10", "Save", and "Run" are present. The "Run" button has been clicked, and the results are shown below the editor:

```
Results Explain Describe Saved SQL History
Total Users: 5
Statement processed.
0.00 seconds
```

The footer of the interface shows "Language: en-us" on the left and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved." on the right.

Question 2 :

Write a PL/SQL function named `get_email` that takes a student ID as input and returns the email address of that student from the `users` table. Also write an anonymous PL/SQL block to call the function and display the email using `DBMS_OUTPUT`?

Answer :

```
CREATE OR REPLACE FUNCTION get_email(p_student_id VARCHAR2) RETURN  
VARCHAR2 IS
```

```
    v_email VARCHAR2(100);
```

```
BEGIN
```

```
    SELECT email INTO v_email FROM users WHERE student_id = p_student_id;
```

```
    RETURN v_email;
```

```
END;
```

```
/
```

Test Function 2

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Email: ' || get_email('24-56434-1'));
```

```
END;
```

```
/
```

The screenshot displays the Oracle Database Express Edition web interface. At the top, the title bar reads "ORACLE Database Express Edition". Below it, the user is logged in as "User AUB PORTAL". The main content area shows the "SQL Commands" tab with the following code entered:

```
CREATE OR REPLACE FUNCTION get_email(p_student_id VARCHAR2) RETURN VARCHAR2 IS  
    v_email VARCHAR2(100);  
BEGIN  
    SELECT email INTO v_email FROM users WHERE student_id = p_student_id;  
    RETURN v_email;  
END;  
/  
  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Email: ' || get_email('24-56434-1'));  
END;  
/
```

The code is executed, and the output is displayed in the "Results" tab below the editor. The output shows the email address: "Email: 24-56434-1@student.aiub.edu". Below the output, it states "Statement processed." and "0.00 seconds".

At the bottom of the interface, the status bar shows "Language: en-us" and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved."

2. Stored Procedures

Question 1:

Write a PL/SQL procedure named print_all_users that retrieves and prints all user full names from the users table using a loop and DBMS_OUTPUT?

Answer :

```
CREATE OR REPLACE PROCEDURE print_all_users IS
```

```
BEGIN
```

```
    FOR rec IN (SELECT full_name FROM users) LOOP
```

```
        DBMS_OUTPUT.PUT_LINE('User: ' || rec.full_name);
```

```
    END LOOP;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    print_all_users;
```

```
END;
```

```
/
```

The screenshot displays the Oracle Database Express Edition web interface. At the top, it says "ORACLE Database Express Edition" and "User: AUBI_PORTAL". Below this is a navigation bar with "Home", "Logout", and "Help" links. The main area is titled "SQL Commands" and contains a text editor with the following PL/SQL code:

```
CREATE OR REPLACE PROCEDURE print_all_users IS
BEGIN
    FOR rec IN (SELECT full_name FROM users) LOOP
        DBMS_OUTPUT.PUT_LINE('User: ' || rec.full_name);
    END LOOP;
END;
/

BEGIN
    print_all_users;
END;
/
```

Below the code editor, there are tabs for "Results", "Explain", "Describe", "Saved SQL", and "History". The "Results" tab is active, showing the output of the procedure:

```
User: John Smith
User: Sarah Johnson
User: Robert Davis
User: Emily Brown
User: Michael Wilson

Statement processed.

0.00 seconds
```

At the bottom of the interface, there is a footer with "Language: en-us" and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved."

Question 2:

Write a PL/SQL procedure named `update_name_edit` that takes a user ID as input and increments the `name_edit_count` column by 1 for that user in the `users` table, then commits the change?

Answer :

```
CREATE OR REPLACE PROCEDURE update_name_edit(p_id NUMBER) IS
```

```
BEGIN
```

```
    UPDATE users SET name_edit_count = name_edit_count + 1 WHERE id = p_id;
```

```
    COMMIT;
```

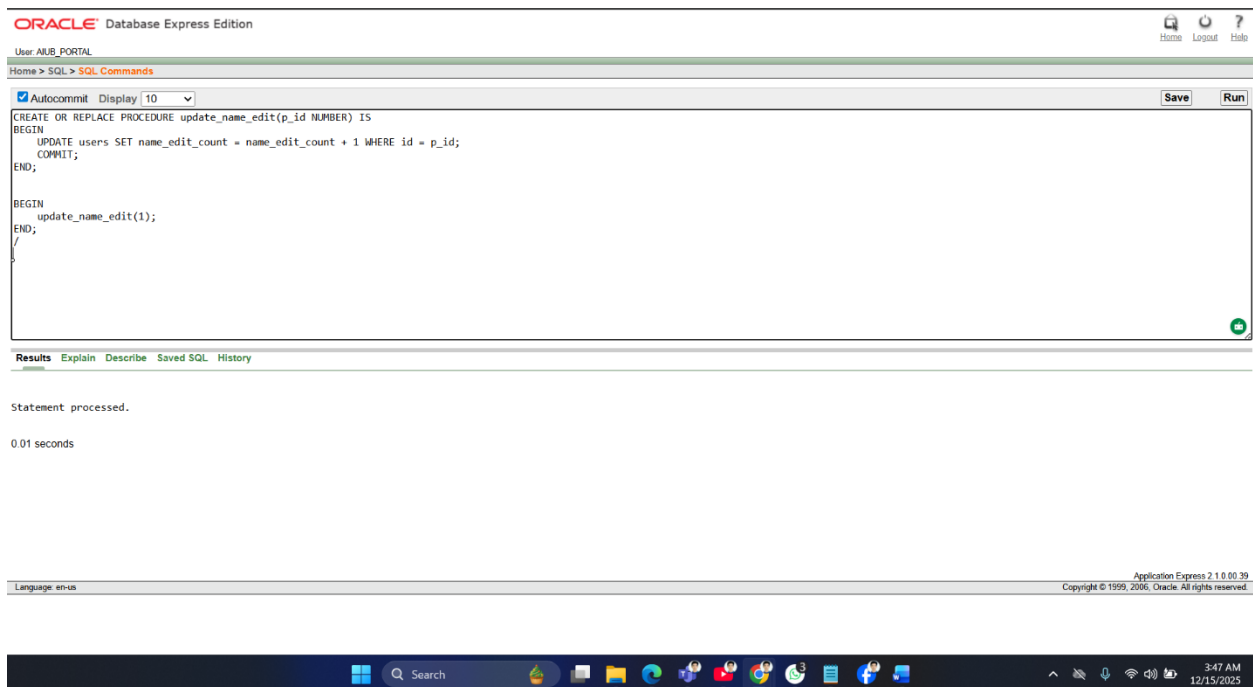
```
END;
```

Test Procedure 2

```
BEGIN
```

```
    update_name_edit(1);
```

```
END;
```



2 Table-Based Record

Question 1:

Write a PL/SQL block that uses %ROWTYPE to retrieve all column values of a user with a specific ID from the users table and display the user's full name and email using DBMS_OUTPUT?

Answer :

DECLARE

 v_user users%ROWTYPE;

BEGIN

 SELECT * INTO v_user FROM users WHERE id = 1;

 DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);

END;

The screenshot displays the Oracle Database Express Edition web interface. At the top, the title bar reads "ORACLE Database Express Edition" with navigation links for Home, Logout, and Help. Below this, the user is identified as "User: AIUB_PORTAL". The main content area is titled "Home > SQL > SQL Commands". It features a text editor with a "Save" and "Run" button. The SQL command entered is:

```
DECLARE
  v_user users%ROWTYPE;
BEGIN
  SELECT * INTO v_user FROM users WHERE id = 1;
  DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);
END;
/
```

Below the editor, the "Results" tab is active, showing the output of the query:

```
Name: John Smith, Email: 24-56434-1@student.aiub.edu
Statement processed.

0.00 seconds
```

The footer of the interface includes the text "Language: en-us" on the left and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved." on the right. At the very bottom, a Windows taskbar is visible with the search bar and various application icons.

Question 2:

Write a PL/SQL block that declares a table of records using `users%ROWTYPE`, retrieves multiple rows from the `users` table using `BULK COLLECT`, and displays the full name of each user using a loop and `DBMS_OUTPUT`?

Answer :

DECLARE

```
TYPE t_users IS TABLE OF users%ROWTYPE;
```

```
v_users t_users;
```

BEGIN

```
SELECT * BULK COLLECT INTO v_users FROM users WHERE id <= 3;
```

```
FOR i IN 1..v_users.COUNT LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('User: ' || v_users(i).full_name);
```

```
END LOOP;
```

END;

The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The main content area displays the SQL Commands tab with the following PL/SQL code:

```
DECLARE
  TYPE t_users IS TABLE OF users%ROWTYPE;
  v_users t_users;
BEGIN
  SELECT * BULK COLLECT INTO v_users FROM users WHERE id <= 3;
  FOR i IN 1..v_users.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('User: ' || v_users(i).full_name);
  END LOOP;
END;
```

Below the code editor, the 'Results' tab shows the output of the execution:

```
User: John Smith
User: Sarah Johnson
User: Robert Davis
Statement processed.
0.01 seconds
```

The bottom of the interface shows the application version 'Application Express 2.1.0.00.39' and the copyright notice 'Copyright © 1999, 2006, Oracle. All rights reserved.'

2 Explicit Cursor

Question 1:

Write a PL/SQL block that declares an explicit cursor to retrieve all email addresses from the users table and prints each email using DBMS_OUTPUT?

Answer :

DECLARE

CURSOR c_users IS SELECT email FROM users;

v_email users.email%TYPE;

BEGIN

OPEN c_users;

LOOP

FETCH c_users INTO v_email;

EXIT WHEN c_users%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);

END LOOP;

CLOSE c_users;

END;

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The main content area shows the 'SQL Commands' tab with a text editor containing the PL/SQL code from the previous blocks. The code is as follows:

```
DECLARE
  CURSOR c_users IS SELECT email FROM users;
  v_email users.email%TYPE;
BEGIN
  OPEN c_users;
  LOOP
    FETCH c_users INTO v_email;
    EXIT WHEN c_users%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
  END LOOP;
  CLOSE c_users;
END;
```

Below the code editor, the 'Results' tab is active, showing the output of the query:

```
Email: 20-12345-5@student.aiub.edu
Email: 21-23456-4@student.aiub.edu
Email: 22-34567-3@student.aiub.edu
Email: 23-45123-2@student.aiub.edu
Email: 24-56434-1@student.aiub.edu
```

Below the output, the status 'Statement processed.' and the execution time '0.00 seconds' are displayed. The bottom of the interface shows the 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

Question 2:

Write a PL/SQL block that uses an explicit cursor to count the number of users for each program_level from the users table and display the program level with its corresponding user count using DBMS_OUTPUT?

Answer :

DECLARE

```
CURSOR c_prog IS SELECT program_level, COUNT(*) AS cnt FROM users GROUP BY program_level;
```

```
v_prog c_prog%ROWTYPE;
```

BEGIN

```
OPEN c_prog;
```

LOOP

```
FETCH c_prog INTO v_prog;
```

```
EXIT WHEN c_prog%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE(v_prog.program_level || ': ' || v_prog.cnt);
```

```
END LOOP;
```

```
CLOSE c_prog;
```

END;

The screenshot shows the Oracle Database Express Edition web interface. At the top, it says "ORACLE Database Express Edition" and "User: AUB_PORTAL". Below that, there's a navigation bar with "Home", "Logout", and "Help". The main area is titled "SQL Commands" and contains a text editor with the following PL/SQL code:

```
DECLARE
  CURSOR c_prog IS SELECT program_level, COUNT(*) AS cnt FROM users GROUP BY program_level;
  v_prog c_prog%ROWTYPE;
BEGIN
  OPEN c_prog;
  LOOP
    FETCH c_prog INTO v_prog;
    EXIT WHEN c_prog%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_prog.program_level || ': ' || v_prog.cnt);
  END LOOP;
  CLOSE c_prog;
END;
```

Below the code editor, there are buttons for "Autocommit", "Display 10", "Save", and "Run". The "Run" button has been clicked, and the results are displayed below. The results show:

```
Undergraduate: 3
Postgraduate: 2
Statement processed.
0.01 seconds
```

At the bottom of the interface, there's a footer with "Language: en-us" and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved."

2 Cursor-Based Record

Question 1:

Write a PL/SQL block that declares an explicit cursor with a record to retrieve full_name and email from the users table and displays each user's name and email using DBMS_OUTPUT?

Answer :

DECLARE

```
CURSOR c_users IS SELECT full_name, email FROM users;
```

```
v_user c_users%ROWTYPE;
```

BEGIN

```
OPEN c_users;
```

LOOP

```
FETCH c_users INTO v_user;
```

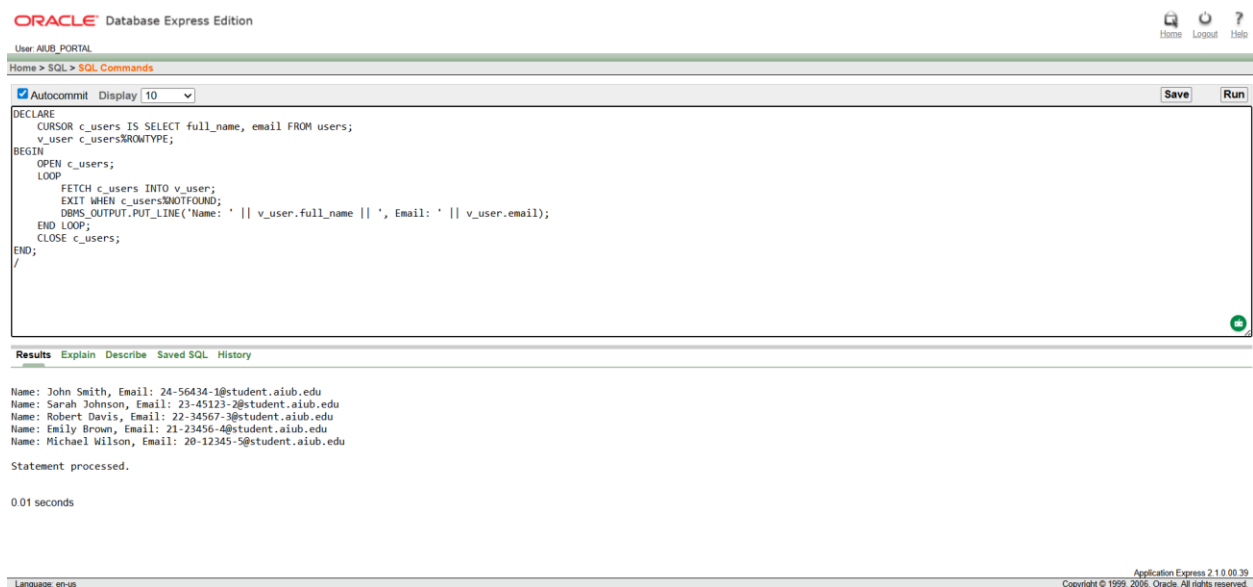
```
EXIT WHEN c_users%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);
```

```
END LOOP;
```

```
CLOSE c_users;
```

END;



The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The main content area displays the SQL Commands window with the following PL/SQL code:

```
DECLARE
  CURSOR c_users IS SELECT full_name, email FROM users;
  v_user c_users%ROWTYPE;
BEGIN
  OPEN c_users;
  LOOP
    FETCH c_users INTO v_user;
    EXIT WHEN c_users%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);
  END LOOP;
  CLOSE c_users;
END;
```

Below the code window, the 'Results' tab is active, showing the output of the query:

```
Name: John Smith, Email: 24-56434-1@student.aiub.edu
Name: Sarah Johnson, Email: 23-45123-2@student.aiub.edu
Name: Robert Davis, Email: 22-34567-3@student.aiub.edu
Name: Emily Brown, Email: 21-23456-4@student.aiub.edu
Name: Michael Wilson, Email: 20-12345-5@student.aiub.edu
```

The status bar at the bottom indicates 'Statement processed.' and '0.01 seconds'.

Question 2:

Write a PL/SQL block that declares a parameterized cursor to retrieve and display the full names of students from the users table for a given department (e.g., CSE) using DBMS_OUTPUT?

Answer :

DECLARE

CURSOR c_students(p_dept VARCHAR2) IS SELECT full_name FROM users WHERE
department = p_dept;

v_user c_students%ROWTYPE;

BEGIN

OPEN c_students('CSE');

LOOP

FETCH c_students INTO v_user;

EXIT WHEN c_students%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('CSE Student: ' || v_user.full_name);

END LOOP;

CLOSE c_students;

END;

The screenshot displays the Oracle Database Express Edition web interface. At the top, it shows 'ORACLE Database Express Edition' and user information 'User: AUB_PORTAL'. The main area is titled 'SQL Commands' and contains a text editor with the following PL/SQL code:

```
DECLARE
  CURSOR c_students(p_dept VARCHAR2) IS SELECT full_name FROM users WHERE department = p_dept;
  v_user c_students%ROWTYPE;
BEGIN
  OPEN c_students('CSE');
  LOOP
    FETCH c_students INTO v_user;
    EXIT WHEN c_students%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('CSE Student: ' || v_user.full_name);
  END LOOP;
  CLOSE c_students;
END;
```

Below the code editor, the 'Results' tab is active, showing the output: 'CSE Student: John Smith'. Below the output, it states 'Statement processed.' and '0.02 seconds'. The bottom of the interface shows the 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

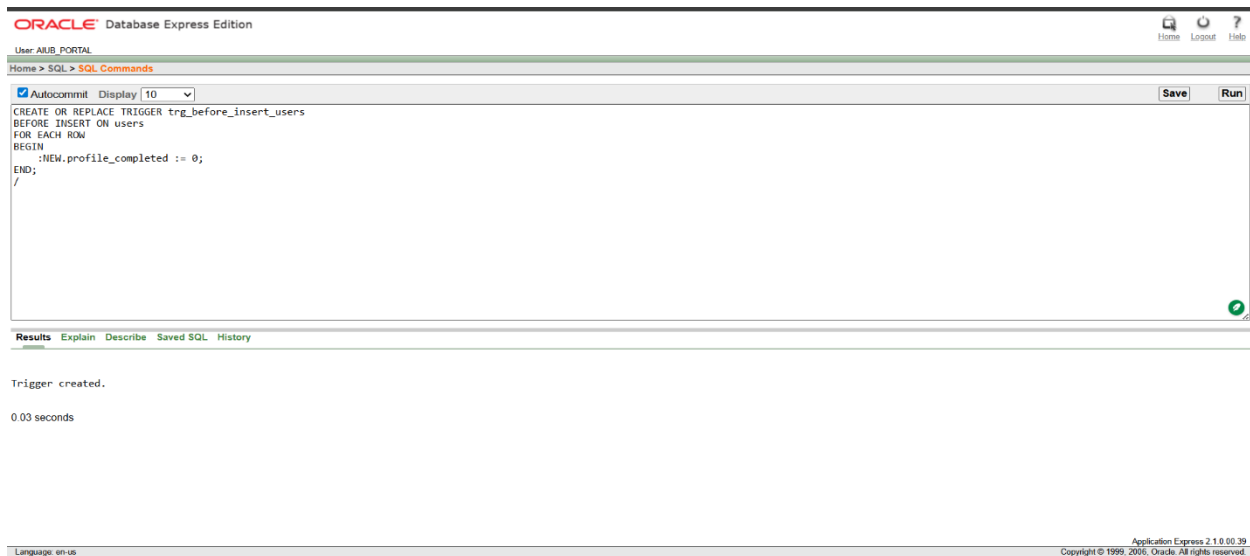
2 Row-Level Trigger

Question 1:

Create a BEFORE INSERT trigger on the users table that sets the profile_completed column to 0 for each new row?

Answer :

```
CREATE OR REPLACE TRIGGER trg_before_insert_users
BEFORE INSERT ON users
FOR EACH ROW
BEGIN
    :NEW.profile_completed := 0;
END;
```



Question 2:

Create a BEFORE UPDATE trigger on the users table that increments the name_edit_count column by 1 whenever the full_name of a user is updated?

Answer :

```
CREATE OR REPLACE TRIGGER trg_before_update_name
```

```
BEFORE UPDATE OF full_name ON users
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    :NEW.name_edit_count := :OLD.name_edit_count + 1;
```

```
END;
```

The screenshot displays the Oracle Database Express Edition web interface. At the top, it shows 'ORACLE Database Express Edition' and user information 'User AUB_PORTAL'. The main area is titled 'SQL Commands' and contains the following SQL code:

```
CREATE OR REPLACE TRIGGER trg_before_update_name
BEFORE UPDATE OF full_name ON users
FOR EACH ROW
BEGIN
    :NEW.name_edit_count := :OLD.name_edit_count + 1;
END;
/
```

Below the code editor, the status 'Trigger created.' is displayed, along with the execution time '0.00 seconds'. The bottom of the interface shows the 'Application Express 2.1.0.00.39' version and copyright information.



2Statement-Level Trigger

Question 1:

Create an AFTER INSERT trigger on the users table that displays a message using DBMS_OUTPUT whenever a new user is inserted?

Answer :

```
CREATE OR REPLACE TRIGGER trg_after_insert_users
AFTER INSERT ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A new user has been inserted.');
```

END;

The screenshot displays the Oracle Database Express Edition web interface. At the top, the title bar reads "ORACLE Database Express Edition" with navigation links for Home, Logout, and Help. Below this, the breadcrumb trail shows "Home > SQL > SQL Commands". The main editor area contains the following SQL code:

```
CREATE OR REPLACE TRIGGER trg_after_insert_users
AFTER INSERT ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A new user has been inserted.');
```

The code is enclosed in a text box with "Autocommit" checked and "Display" set to 10. A green checkmark icon in the bottom right corner of the editor indicates successful execution. Below the editor, a tabbed interface shows "Results" as the active tab, displaying the message "Trigger created." and a duration of "0.00 seconds". The footer of the application shows "Language: en-us" and "Application Express 2.1.10.00.39 Copyright © 1999, 2006, Oracle. All rights reserved." The Windows taskbar at the bottom shows the time as 3:59 AM on 12/15/2025.

Question 2:

Create an AFTER DELETE trigger on the users table that displays a message using DBMS_OUTPUT whenever a user is deleted?

Answer :

```
CREATE OR REPLACE TRIGGER trg_after_delete_users
```

```
AFTER DELETE ON users
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('A user has been deleted.');
```

```
END;
```

The screenshot shows the Oracle Database Express Edition web interface. At the top, it says "ORACLE Database Express Edition" and "User AUB_PORTAL". Below the navigation bar, there's a breadcrumb "Home > SQL > SQL Commands". The main area has a text editor with the following SQL code:

```
CREATE OR REPLACE TRIGGER trg_after_delete_users
AFTER DELETE ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A user has been deleted.');
```

The code is entered into the editor, and the "Run" button is visible. Below the editor, the "Results" tab is selected, showing the message "Trigger created." and the execution time "0.01 seconds". The footer of the interface includes "Language: en-us" and "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved."

2 Package

Question 1:

Create a PL/SQL package named pkg_users that includes a procedure print_user to display a user's name by ID and a function get_user_email to return the user's email by ID, then demonstrate calling both from an anonymous block?

Answer :

```
CREATE OR REPLACE PACKAGE pkg_users IS
```

```
    PROCEDURE print_user(p_id NUMBER);
```

```
    FUNCTION get_user_email(p_id NUMBER) RETURN VARCHAR2;
```

```
END pkg_users;
```

```
/
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY pkg_users IS
```

```
    PROCEDURE print_user(p_id NUMBER) IS
```

```
        v_name VARCHAR2(100);
```

```
    BEGIN
```

```
        SELECT full_name INTO v_name FROM users WHERE id = p_id;
```

```
        DBMS_OUTPUT.PUT_LINE('User Name: ' || v_name);
```

```
    END print_user;
```

```
    FUNCTION get_user_email(p_id NUMBER) RETURN VARCHAR2 IS
```

```
        v_email VARCHAR2(100);
```

```
    BEGIN
```

```
        SELECT email INTO v_email FROM users WHERE id = p_id;
```

```
        RETURN v_email;
```

```
    END get_user_email;
```

```
END pkg_users;
```

Test Package

```
BEGIN
```

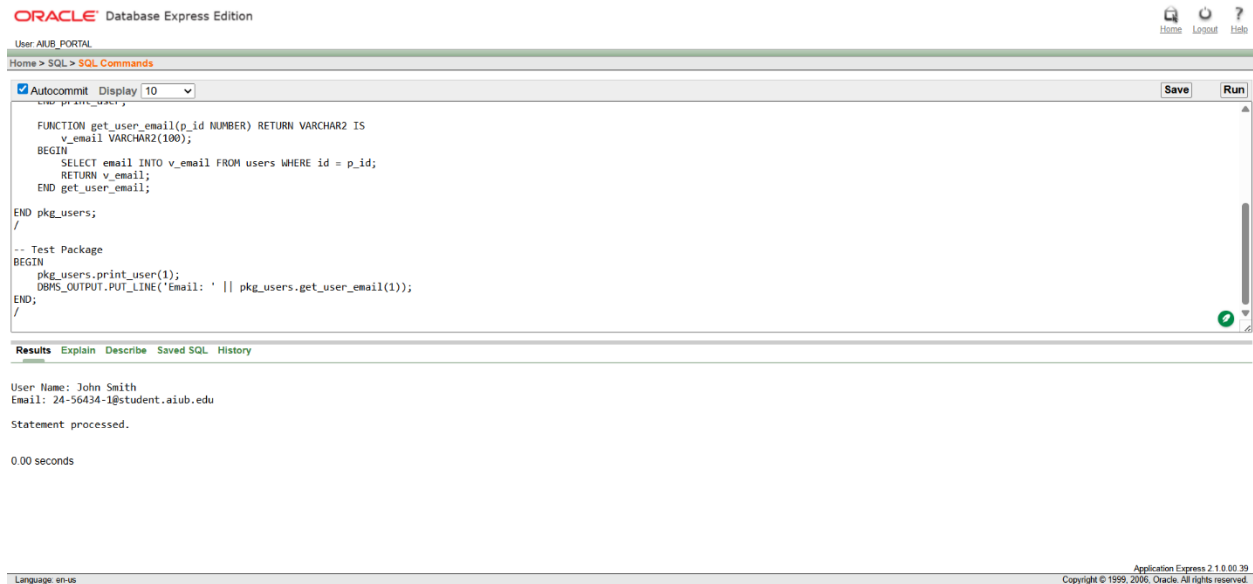
```

pkg_users.print_user(1);

DBMS_OUTPUT.PUT_LINE('Email: ' || pkg_users.get_user_email(1));

END;

```



Question 2:

Create a PL/SQL package named `pkg_users_info` with a procedure `print_user_department` to display a user's department by ID and a function `get_user_phone` to return the user's phone number by ID, then demonstrate calling both from an anonymous block?

Answer :

```

CREATE OR REPLACE PACKAGE pkg_users_info IS
  PROCEDURE print_user_department(p_id NUMBER);
  FUNCTION get_user_phone(p_id NUMBER) RETURN VARCHAR2;
END pkg_users_info;

/

CREATE OR REPLACE PACKAGE BODY pkg_users_info IS
  PROCEDURE print_user_department(p_id NUMBER) IS

```

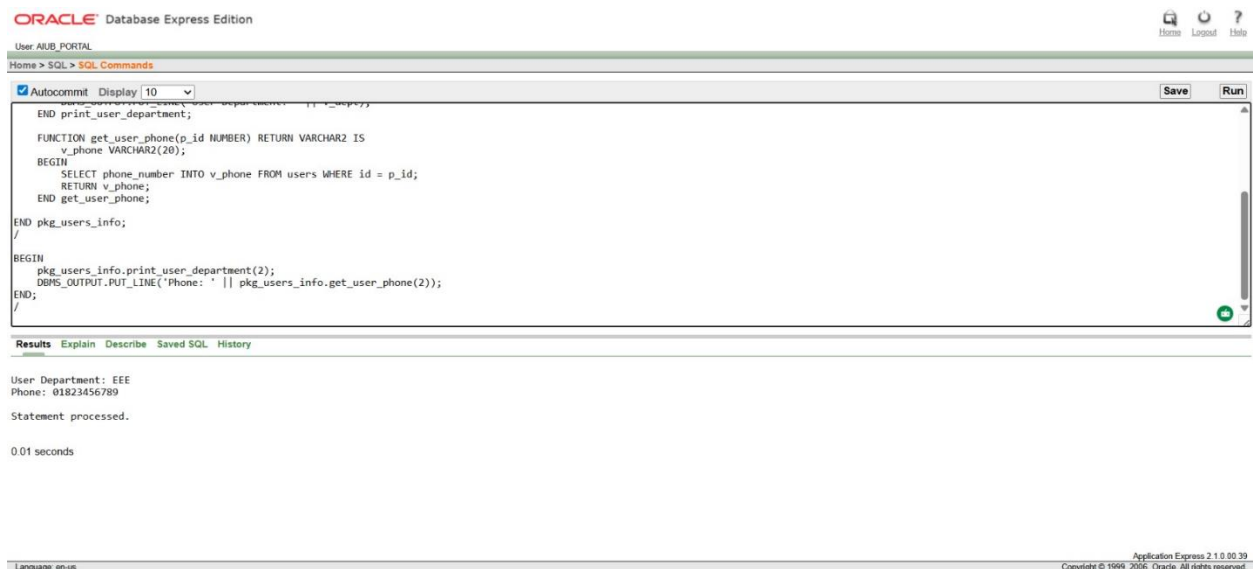
```

    v_dept VARCHAR2(50);
BEGIN
    SELECT department INTO v_dept FROM users WHERE id = p_id;
    DBMS_OUTPUT.PUT_LINE('User Department: ' || v_dept);
END print_user_department;

FUNCTION get_user_phone(p_id NUMBER) RETURN VARCHAR2 IS
    v_phone VARCHAR2(20);
BEGIN
    SELECT phone_number INTO v_phone FROM users WHERE id = p_id;
    RETURN v_phone;
END get_user_phone;
END pkg_users_info;
/

BEGIN
    pkg_users_info.print_user_department(2);
    DBMS_OUTPUT.PUT_LINE('Phone: ' || pkg_users_info.get_user_phone(2));
END;

```



Query Writing

2Stored functions:

Question 1:

Write a function get_total_users that:

- Returns the total number of users.
- Handles exceptions properly:
 - If there is no data, return 0.
 - If any other error occurs, print the error message and return -1.

Answer :

```
CREATE OR REPLACE FUNCTION get_total_users RETURN NUMBER IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM users;
    RETURN v_count;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in get_total_users: ' || SQLERRM);
        RETURN -1;
END;
```

Test the function with exception handling:

```
BEGIN
    DECLARE
        v_total NUMBER;
    BEGIN
        v_total := get_total_users;
```

```
DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_total);
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('Error while fetching total users: ' || SQLERRM);
```

```
END;
```

```
END;
```

The screenshot displays the Oracle Database Express Edition web interface. At the top, the title bar reads "ORACLE Database Express Edition". Below it, the user is identified as "User: AUB PORTAL". The main content area shows the "SQL Commands" tab with a text editor containing the following SQL code:

```
CREATE OR REPLACE FUNCTION get_total_users RETURN NUMBER IS
v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM users;
    RETURN v_count;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in get_total_users: ' || SQLERRM);
        RETURN -1;
END;
/
```

Below the code editor, the "Results" tab is active, showing the output of the SQL execution:

```
Total Users: 5
Statement processed.
0.00 seconds
```

The bottom status bar indicates the application version as "Application Express 2.1.10.00.39" and the copyright as "Copyright © 1999, 2006, Oracle. All rights reserved." The Windows taskbar at the very bottom shows the time as "1:16 AM 1/16/2026".

Question 2:

Write a function get_email that:

- Accepts a parameter p_student_id of type VARCHAR2.
- Returns the email of the student.
- Handles exceptions properly:
 - If no student is found, return 'No email found'.
 - If multiple students are found with the same ID, return 'Multiple emails **found**'.

Answer :

```
CREATE OR REPLACE FUNCTION get_email(p_student_id VARCHAR2) RETURN  
VARCHAR2 IS
```

```
    v_email VARCHAR2(100);
```

```
BEGIN
```

```
    SELECT email INTO v_email FROM users WHERE student_id = p_student_id;
```

```
    RETURN v_email;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        RETURN 'No email found';
```

```
    WHEN TOO_MANY_ROWS THEN
```

```
        RETURN 'Multiple emails found';
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Error in get_email: ' || SQLERRM);
```

```
        RETURN 'Error occurred';
```

```
END;
```

Test the function with exception handling:

```
BEGIN
```

```
    DECLARE
```

```
        v_email VARCHAR2(100);
```

BEGIN

 v_email := get_email('24-56434-1');

 DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);

EXCEPTION

 WHEN OTHERS THEN

 DBMS_OUTPUT.PUT_LINE('Error while fetching email: ' || SQLERRM);

END;

END;

The screenshot displays the Oracle Database Express Edition web interface. At the top, the header shows 'ORACLE Database Express Edition' and navigation links for 'Home', 'Logout', and 'Help'. Below the header, the user is logged in as 'User: AIUB PORTAL'. The main area is titled 'SQL Commands' and contains a text editor with a PL/SQL block. The code defines a variable v_email, calls a function get_email, and uses DBMS_OUTPUT.PUT_LINE to display the email. It also includes an exception handler for 'OTHERS'. The 'Run' button is visible in the top right of the editor. Below the editor, the 'Results' tab is active, showing the output: 'Email: 24-56434-1@student.aiub.edu' and 'Statement processed.' The execution time is listed as '0.02 seconds'. The footer of the interface includes 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

```
SQL> WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in get_email: ' || SQLERRM);
        RETURN 'Error occurred';
END;
/
BEGIN
  DECLARE
    v_email VARCHAR2(100);
  BEGIN
    v_email := get_email('24-56434-1');
    DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
  EXCEPTION
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error while fetching email: ' || SQLERRM);
  END;
END;
/
```

Results Explain Describe Saved SQL History

Email: 24-56434-1@student.aiub.edu
Statement processed.
0.02 seconds

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

2. Stored Procedures

Question 1:

Write a PL/SQL procedure print_all_users that:

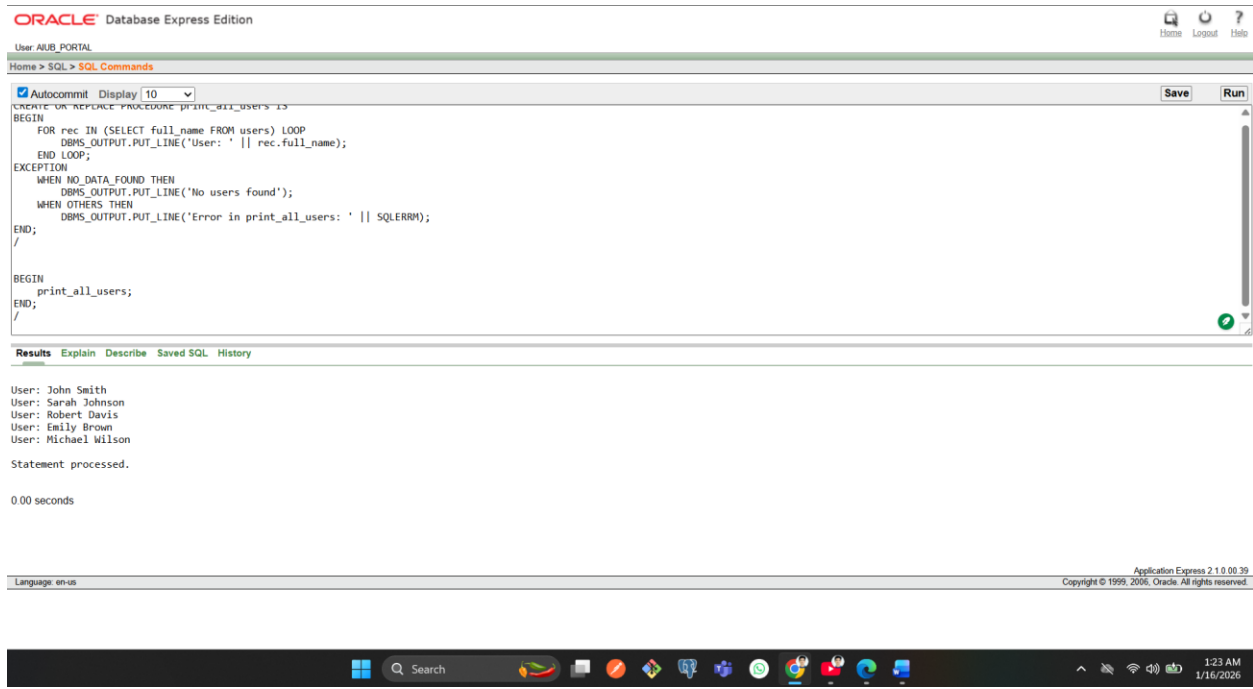
1. Loops through all users in the users table.
2. Prints the full_name of each user using DBMS_OUTPUT.PUT_LINE.
3. Handles exceptions properly:
 - If no users are found, print 'No users found'.
 - For any other error, print the error message.

Answer :

```
CREATE OR REPLACE PROCEDURE print_all_users IS
BEGIN
    FOR rec IN (SELECT full_name FROM users) LOOP
        DBMS_OUTPUT.PUT_LINE('User: ' || rec.full_name);
    END LOOP;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No users found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in print_all_users: ' || SQLERRM);
END;
/
```

Test the function with exception handling:

```
BEGIN
__print_all_users;
END;
/
```



Question 2:

2. Write a PL/SQL procedure `update_name_edit` that:

1. Accepts a parameter `p_id` (user ID) of type `NUMBER`.
2. Updates the column `name_edit_count` of the `users` table by adding 1 for the user with the given ID.
3. Commits the change.
4. Handles exceptions properly:
 - If no user is found with the given ID, print 'User not found'.
 - For any other error, print the error message and roll back the transaction.

Answer :

```

CREATE OR REPLACE PROCEDURE update_name_edit(p_id NUMBER) IS
BEGIN

```

```

  UPDATE users

```

```

  SET name_edit_count = name_edit_count + 1

```

```

WHERE id = p_id;

IF SQL%ROWCOUNT = 0 THEN

    DBMS_OUTPUT.PUT_LINE('User not found');

ELSE

    DBMS_OUTPUT.PUT_LINE('Name edit count updated successfully');

END IF;

COMMIT;

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error in update_name_edit: ' || SQLERRM);

        ROLLBACK;

END;

```

Test the function with exception handling:

```

BEGIN

    update_name_edit(1);

END;

```



2 Table-Based Record

Question 1:

Write a PL/SQL anonymous block that:

1. Declares a variable v_user of type users%ROWTYPE.
2. Selects all columns of the user with id = 1 into v_user.
3. Prints the full_name and email of the user using DBMS_OUTPUT.PUT_LINE.
4. Handles exceptions properly:
 - If no user is found, print 'User not found'.
 - For any other error, print the error message.

Answer :

DECLARE

 v_user users%ROWTYPE;

BEGIN

 SELECT * INTO v_user FROM users WHERE id = 1;

 DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);

EXCEPTION

 WHEN NO_DATA_FOUND THEN

 DBMS_OUTPUT.PUT_LINE('User not found');

 WHEN TOO_MANY_ROWS THEN

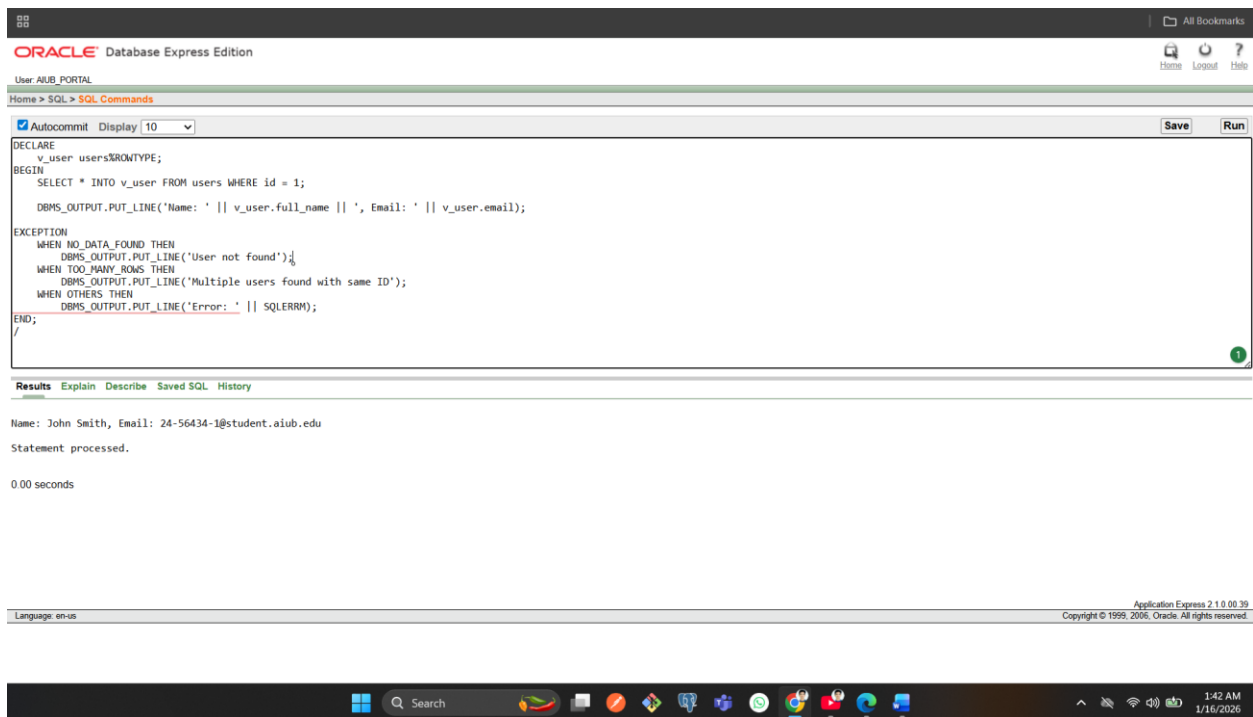
 DBMS_OUTPUT.PUT_LINE('Multiple users found with same ID');

 WHEN OTHERS THEN

 DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

/



Question 2:

Write a PL/SQL anonymous block that:

1. Declares a collection type `t_users` to hold multiple rows of the users table.
2. Declares a variable `v_users` of this type.
3. Uses `BULK COLLECT` to fetch all users with `id <= 3` into `v_users`.
4. Loops through the collection and prints each user's `full_name`.
5. Handles exceptions properly:
 - If no users are found, print 'No users found'.
 - For any other error, print the error message.

Answer :

DECLARE

TYPE `t_users` IS TABLE OF `users%ROWTYPE`;

`v_users` `t_users`;

BEGIN

```

SELECT * BULK COLLECT INTO v_users

FROM users

WHERE id <= 3;

IF v_users.COUNT = 0 THEN

    DBMS_OUTPUT.PUT_LINE('No users found');

ELSE

    FOR i IN 1..v_users.COUNT LOOP

        DBMS_OUTPUT.PUT_LINE('User: ' || v_users(i).full_name);

    END LOOP;

END IF;

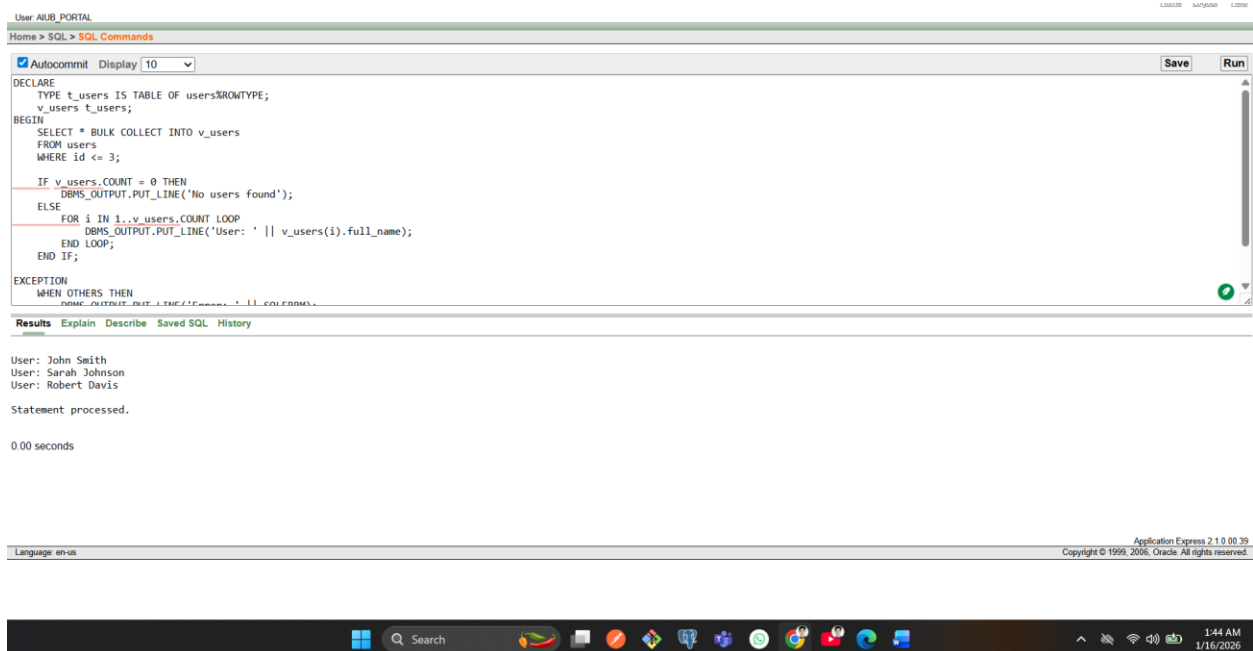
EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

```



2Explicit Cursor

Question 1:

Write a PL/SQL anonymous block that:

1. Declares a cursor `c_users` to select all email values from the users table.
2. Declares a variable `v_email` to store the email fetched from the cursor.
3. Opens the cursor, loops through all rows, and prints each email using `DBMS_OUTPUT.PUT_LINE`.
4. Handles exceptions properly:
 - If no users are found, print 'No emails found'.
 - For any other error, print the error message.

Answer :

```
DECLARE
```

```
    CURSOR c_users IS SELECT email FROM users;
```

```
    v_email users.email%TYPE;
```

```
BEGIN
```

```
    OPEN c_users;
```

```
    LOOP
```

```
        FETCH c_users INTO v_email;
```

```
        EXIT WHEN c_users%NOTFOUND;
```

```
        DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
```

```
    END LOOP;
```

```
    CLOSE c_users;
```

```
EXCEPTION
```

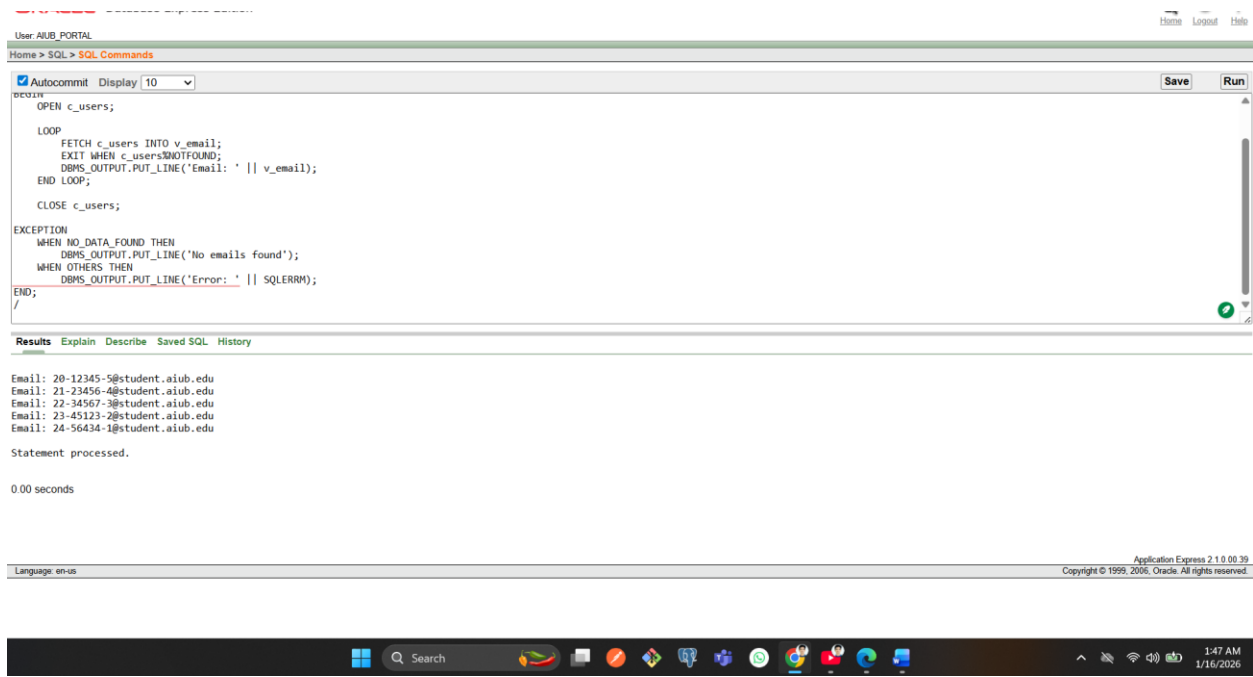
```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('No emails found');
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
```

END;



Question 2:

Write a PL/SQL anonymous block that:

1. Declares a cursor `c_prog` to select `program_level` and the count of users in each level from the `users` table (use `GROUP BY program_level`).
2. Declares a variable `v_prog` to store the cursor row.
3. Opens the cursor, loops through all rows, and prints each `program_level` along with its user count using `DBMS_OUTPUT.PUT_LINE`.
4. Handles exceptions properly:
 - If no rows are found, print 'No programs found'.
 - For any other error, print the error message.

Answer :

DECLARE

CURSOR `c_prog` IS

SELECT `program_level`, COUNT(*) AS `cnt`

FROM `users`

```

        GROUP BY program_level;

v_prog c_prog%ROWTYPE;

BEGIN

    OPEN c_prog;

    LOOP

        FETCH c_prog INTO v_prog;

        EXIT WHEN c_prog%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(v_prog.program_level || ': ' || v_prog.cnt);

    END LOOP;

    CLOSE c_prog;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('No programs found');

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
```

User: AUB_PORTAL Home Logout Help

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

```

OPEN c_prog;

LOOP
    FETCH c_prog INTO v_prog;
    EXIT WHEN c_prog%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_prog.program_level || ': ' || v_prog.cnt);
END LOOP;

CLOSE c_prog;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No programs found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/
```

Results Explain Describe Saved SQL History

Undergraduate: 3
Postgraduate: 2
Statement processed.
0.01 seconds

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2 Cursor-Based Record

Question 1:

Write a PL/SQL anonymous block that:

1. Declares a cursor `c_users` to select `full_name` and `email` from the `users` table.
2. Declares a variable `v_user` to store the cursor row.
3. Opens the cursor, loops through all rows, and prints each user's name and email using `DBMS_OUTPUT.PUT_LINE`.
4. Handles exceptions properly:
 - If no users are found, print 'No users found'.
 - For any other error, print the error message.

Answer :

```
DECLARE
```

```
    CURSOR c_users IS
```

```
        SELECT full_name, email FROM users;
```

```
    v_user c_users%ROWTYPE;
```

```
BEGIN
```

```
    OPEN c_users;
```

```
    LOOP
```

```
        FETCH c_users INTO v_user;
```

```
        EXIT WHEN c_users%NOTFOUND;
```

```
        DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);
```

```
    END LOOP;
```

```
    CLOSE c_users;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('No users found');
```

```
    WHEN OTHERS THEN
```

```

DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

```

The screenshot displays the Oracle SQL Developer environment. The main window shows a PL/SQL anonymous block with the following code:

```

-- cursor
OPEN c_users;

LOOP
    FETCH c_users INTO v_user;
    EXIT WHEN c_users%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);
END LOOP;

CLOSE c_users;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No users found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

Below the code editor, the 'Results' tab shows the output of the execution:

```

Name: John Smith, Email: 24-56434-1@student.aiub.edu
Name: Sarah Johnson, Email: 23-45123-2@student.aiub.edu
Name: Robert Davis, Email: 22-34567-3@student.aiub.edu
Name: Emily Brown, Email: 21-23456-4@student.aiub.edu
Name: Michael Wilson, Email: 20-12345-5@student.aiub.edu

```

The status bar at the bottom indicates 'Statement processed.' and '0.00 seconds'.

Question 2:

Write a PL/SQL anonymous block that:

1. Declares a parameterized cursor `c_students(p_dept VARCHAR2)` to select `full_name` from the `users` table for a given department.
2. Declares a variable `v_user` to store the cursor row.
3. Opens the cursor for the department 'CSE', loops through all rows, and prints each student's name using `DBMS_OUTPUT.PUT_LINE`.
4. Handles exceptions properly:
 - If no students are found, print 'No students found in the department'.
 - For any other error, print the error message.

Answer :

DECLARE

CURSOR c_students(p_dept VARCHAR2) IS

SELECT full_name FROM users WHERE department = p_dept;

v_user c_students%ROWTYPE;

BEGIN

OPEN c_students('CSE');

LOOP

FETCH c_students INTO v_user;

EXIT WHEN c_students%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('CSE Student: ' || v_user.full_name);

END LOOP;

CLOSE c_students;

EXCEPTION

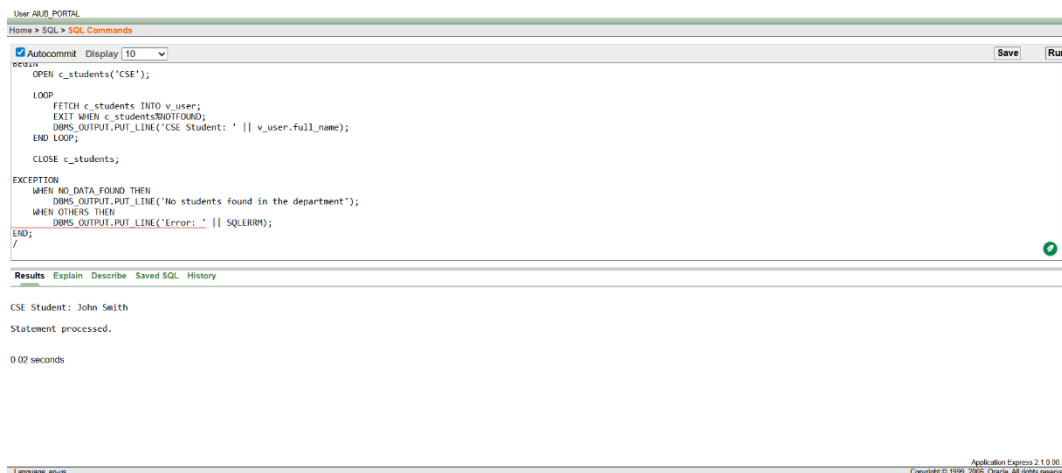
WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('No students found in the department');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;



```
SQL> DECLARE
CURSOR c_students(p_dept VARCHAR2) IS
SELECT full_name FROM users WHERE department = p_dept;
v_user c_students%ROWTYPE;
BEGIN
OPEN c_students('CSE');
LOOP
FETCH c_students INTO v_user;
EXIT WHEN c_students%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('CSE Student: ' || v_user.full_name);
END LOOP;
CLOSE c_students;
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No students found in the department');
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Results Explain Describe Saved SQL History

CSE Student: John Smith
Statement processed.
0.02 seconds

Language: en-us

2 Row-Level Trigger

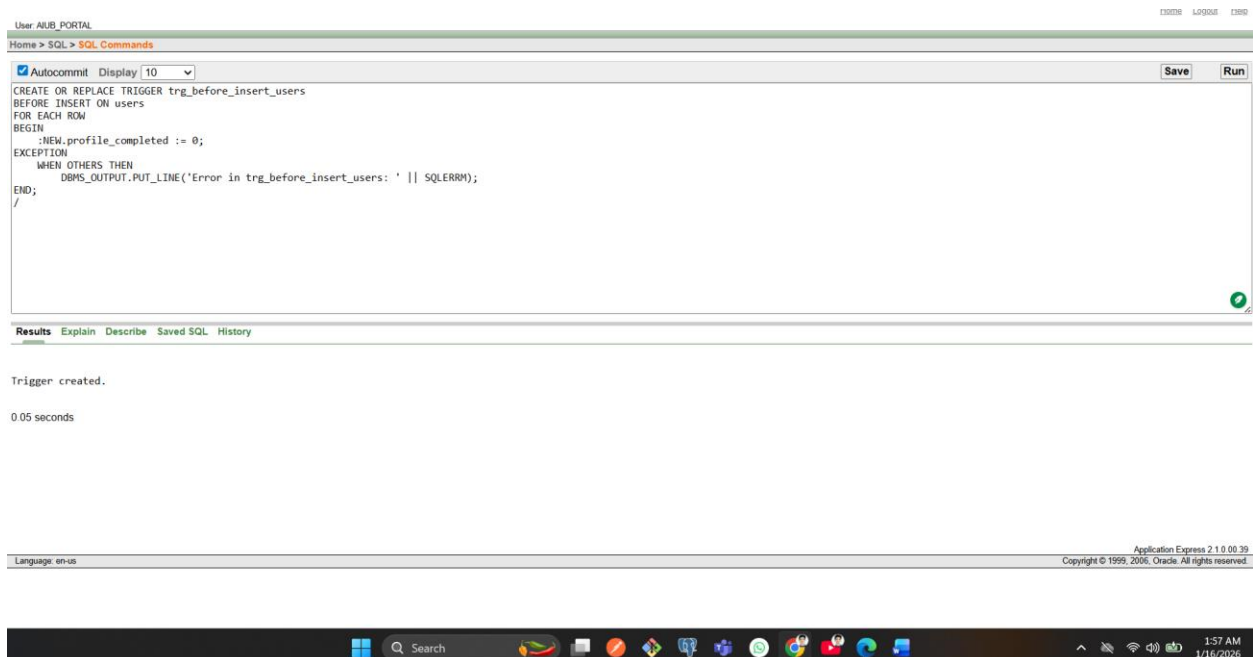
Question 1:

Write a PL/SQL BEFORE INSERT trigger that:

1. Automatically sets the profile_completed column to 0 whenever a new row is inserted into the users table.
2. Handles exceptions by printing the error message.

Answer :

```
CREATE OR REPLACE TRIGGER trg_before_insert_users
BEFORE INSERT ON users
FOR EACH ROW
BEGIN
    :NEW.profile_completed := 0;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in trg_before_insert_users: ' || SQLERRM);
END;
```



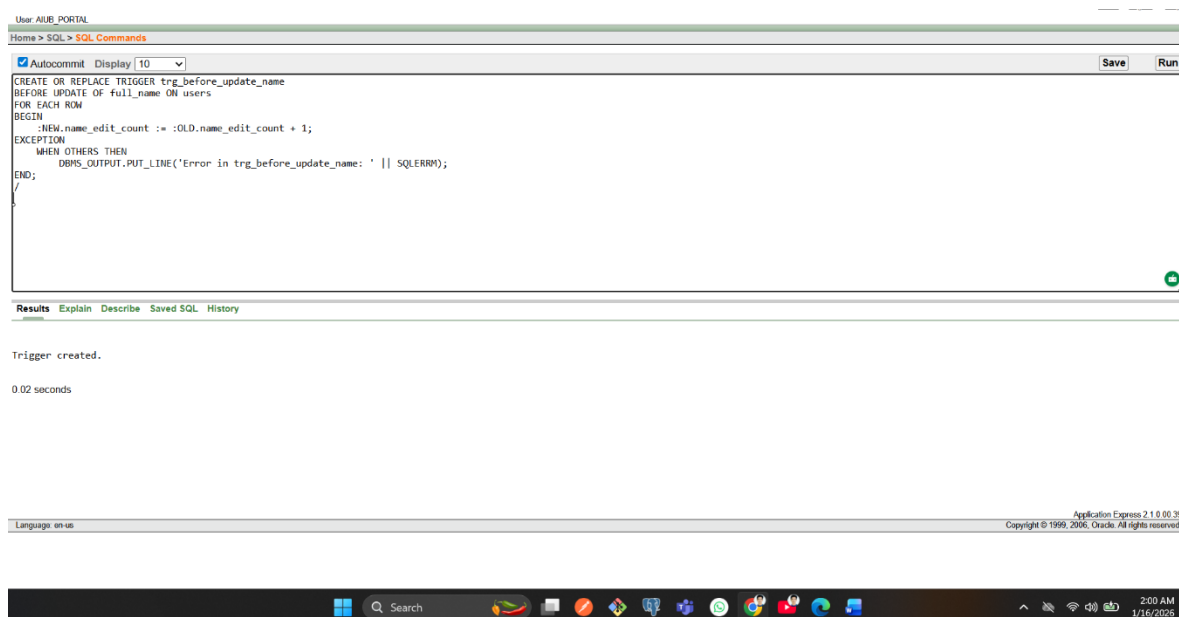
Question 2:

Write a PL/SQL BEFORE UPDATE trigger that:

1. Fires before updating the full_name column of the users table.
2. Automatically increments the name_edit_count column by 1 whenever the full_name is updated.
3. Handles exceptions by printing the error message.

Answer :

```
CREATE OR REPLACE TRIGGER trg_before_update_name
BEFORE UPDATE OF full_name ON users
FOR EACH ROW
BEGIN
    :NEW.name_edit_count := :OLD.name_edit_count + 1;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in trg_before_update_name: ' || SQLERRM);
END;
```



2 Statement-Level Trigger

Question 1:

Write a PL/SQL AFTER INSERT trigger that:

1. Fires after a new row is inserted into the users table.
2. Prints the message 'A new user has been inserted.' using DBMS_OUTPUT.PUT_LINE.
3. Handles exceptions by printing the error message.

Answer :

```
CREATE OR REPLACE TRIGGER trg_after_insert_users
```

```
AFTER INSERT ON users
```

```
BEGIN
```

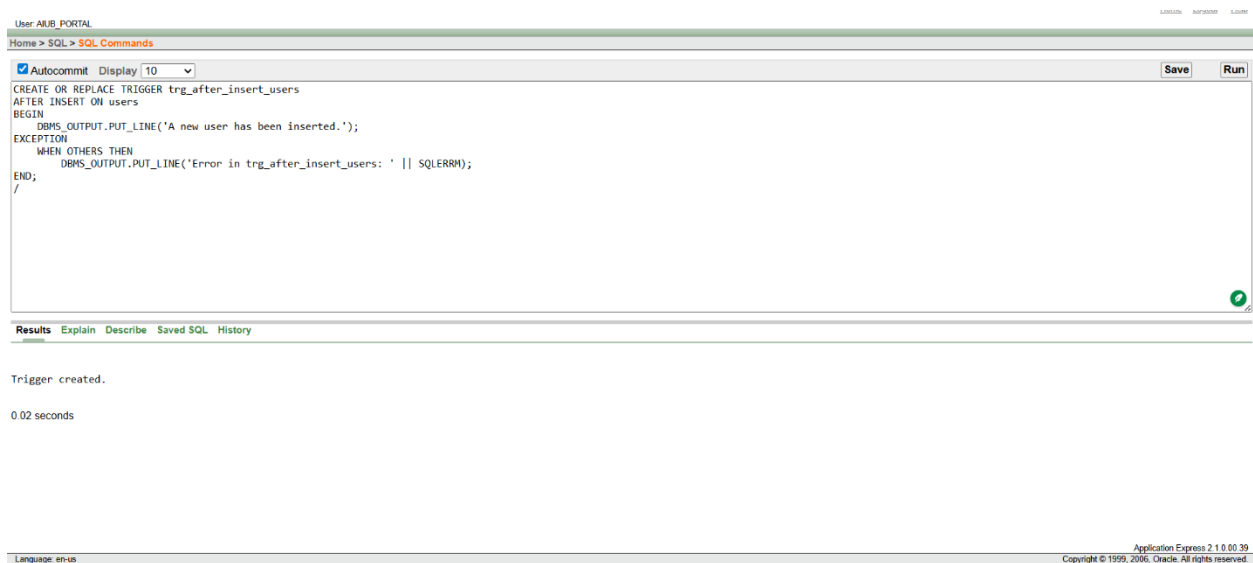
```
    DBMS_OUTPUT.PUT_LINE('A new user has been inserted.');
```

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Error in trg_after_insert_users: ' || SQLERRM);
```

```
END;
```



Question 2:

Write a PL/SQL AFTER DELETE trigger that:

1. Fires after a row is deleted from the users table.
2. Prints the message 'A user has been deleted.' using DBMS_OUTPUT.PUT_LINE.
3. Handles exceptions by printing the error message.

Answer :

```
CREATE OR REPLACE TRIGGER trg_after_delete_users
AFTER DELETE ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A user has been deleted.');
```

EXCEPTION

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in trg_after_delete_users: ' || SQLERRM);
END;
```

Output:

The screenshot displays the SQL Developer application. At the top, the 'SQL Commands' window shows the following code:

```
CREATE OR REPLACE TRIGGER trg_after_delete_users
AFTER DELETE ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A user has been deleted.');
```

Below the code window, the 'Results' window shows the output:

```
Trigger created.
0.02 seconds
```

The bottom of the screenshot shows the Windows taskbar with various application icons and the system clock indicating 2:02 AM on 1/16/2026.

2Package

Question 1:

Create a PL/SQL package pkg_users that:

1. Declares a procedure print_user(p_id NUMBER) to print the full name of a user given their ID.
2. Declares a function get_user_email(p_id NUMBER) that returns the email of a user given their ID.
3. Both the procedure and function should handle exceptions properly:
 - If no user is found, print 'User not found' for the procedure and return 'No email found' for the function.
 - Handle any other errors by printing the error message.

Answer :

1. Package Specification

```
CREATE OR REPLACE PACKAGE pkg_users IS
    PROCEDURE print_user(p_id NUMBER);
    FUNCTION get_user_email(p_id NUMBER) RETURN VARCHAR2;
END pkg_users;
/
```

2. Package Body with Exception Handling

```
CREATE OR REPLACE PACKAGE BODY pkg_users IS

    PROCEDURE print_user(p_id NUMBER) IS
        v_name VARCHAR2(100);
    BEGIN
        SELECT full_name INTO v_name FROM users WHERE id = p_id;
        DBMS_OUTPUT.PUT_LINE('User Name: ' || v_name);
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('User not found');
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error in print_user: ' || SQLERRM);
    END print_user;

    FUNCTION get_user_email(p_id NUMBER) RETURN VARCHAR2 IS
        v_email VARCHAR2(100);
    BEGIN
        SELECT email INTO v_email FROM users WHERE id = p_id;
```

```

        RETURN v_email;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 'No email found';
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error in get_user_email: ' || SQLERRM);
            RETURN 'Error occurred';
        END get_user_email;
    END pkg_users;
/

3. Test the Package
BEGIN
    pkg_users.print_user(1);
    DBMS_OUTPUT.PUT_LINE('Email: ' || pkg_users.get_user_email(1));
END;

```

User AUB_PORTAL

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

```

RETURN 'No email found';
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in get_user_email: ' || SQLERRM);
        RETURN 'Error occurred';
    END get_user_email;
END pkg_users;
/

-- Test the Package
BEGIN
    pkg_users.print_user(1);
    DBMS_OUTPUT.PUT_LINE('Email: ' || pkg_users.get_user_email(1));
END;
/

```

Results Explain Describe Saved SQL History

User Name: John Smith
Email: 24-56434-1@student.aiub.edu
Statement processed.
0.00 seconds

Application: Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Question 2:

Create a PL/SQL package named pkg_users_info that:

1. Declares
 - A procedure print_user_department(p_id NUMBER) to print the department of a user
 - A function get_user_phone(p_id NUMBER) to return the phone number of a user
2. The procedure should:
 - Fetch the department from the users table using id
 - Print the department using DBMS_OUTPUT.PUT_LINE
 - Handle the case where the user does not exist
3. The function should:
 - Return the phone number of the user
 - Handle the case where the user does not exist
 - Handle other unexpected errors
4. Write a PL/SQL block to test the package for user ID 2.

Answer :

```
CREATE OR REPLACE PACKAGE pkg_users_info IS
    PROCEDURE print_user_department(p_id NUMBER);
    FUNCTION get_user_phone(p_id NUMBER) RETURN VARCHAR2;
END pkg_users_info;

/Package Body:

CREATE OR REPLACE PACKAGE BODY pkg_users_info IS
    PROCEDURE print_user_department(p_id NUMBER) IS
        v_dept VARCHAR2(50);
    BEGIN
        SELECT department INTO v_dept FROM users WHERE id = p_id;
        DBMS_OUTPUT.PUT_LINE('User Department: ' || v_dept);
```

```

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('User not found');

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error in print_user_department: ' || SQLERRM);

END print_user_department;

FUNCTION get_user_phone(p_id NUMBER) RETURN VARCHAR2 IS

    v_phone VARCHAR2(20);

BEGIN

    SELECT phone_number INTO v_phone FROM users WHERE id = p_id;

    RETURN v_phone;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RETURN 'No phone number found';

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error in get_user_phone: ' || SQLERRM);

        RETURN 'Error occurred';

END get_user_phone;

END pkg_users_info;

/

Test Block:

BEGIN

    pkg_users_info.print_user_department(2);

    DBMS_OUTPUT.PUT_LINE('Phone: ' || pkg_users_info.get_user_phone(2));

END;

/

```

User: AUB_PORTAL
Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'No phone number found';
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error in get_user_phone: ' || SQLERRM);
    RETURN 'Error occurred';
END get_user_phone;

END pkg_users_info;
/

BEGIN
  pkg_users_info.print_user_department(2);
  DBMS_OUTPUT.PUT_LINE('Phone: ' || pkg_users_info.get_user_phone(2));
END;
/

```

Results Explain Describe Saved SQL History

User Department: EEE
Phone: 01823456789
Statement processed.
0.01 seconds

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

Relational Algebra

Question 1: Find the full names and emails of all Undergraduate students.

Answer: $\Pi_{full_name, email}(\sigma_{program_level="Undergraduate"}(USERS))$

Question 2: Find the full names of students who belong to the CSE department.

Answer: $\Pi_{full_name}(\sigma_{department="CSE"}(USERS))$

Question 3: Find the student IDs and phone numbers of Female students.

Answer: $\Pi_{student_id, phone_number}(\sigma_{gender="Female"}(USERS))$

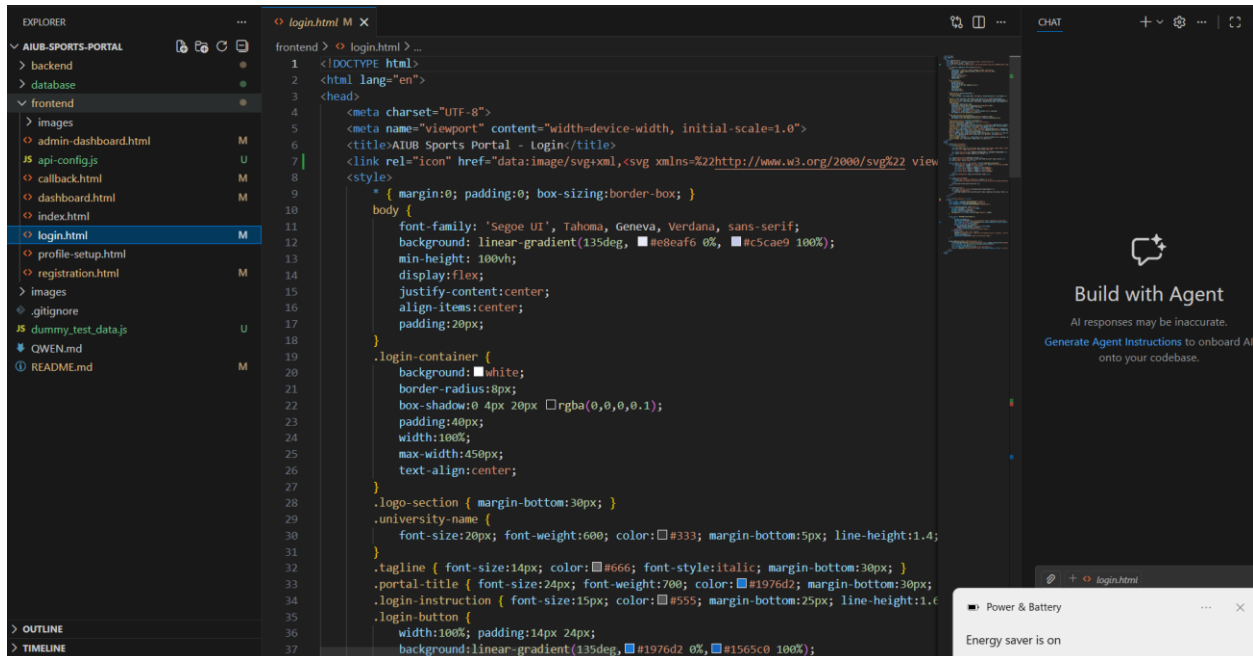
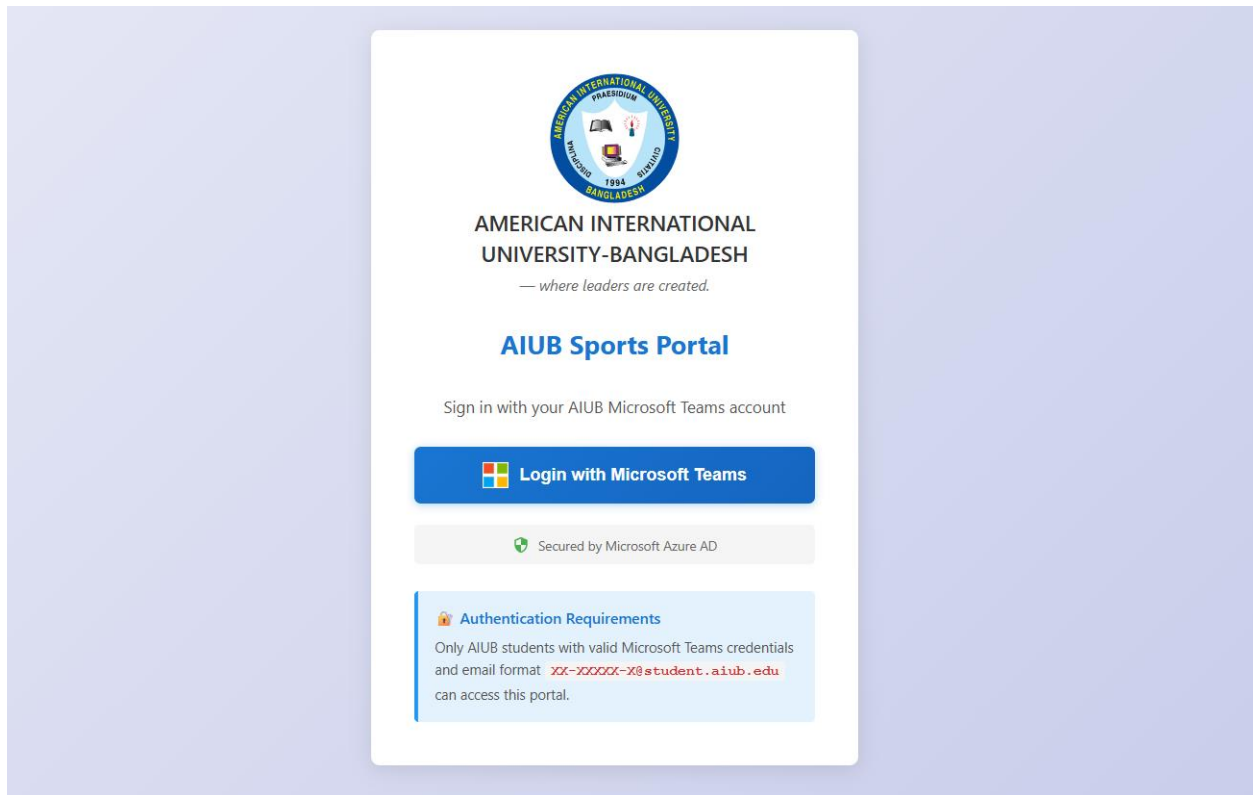
Question 4: Find the full names of students whose name has been edited more than once.

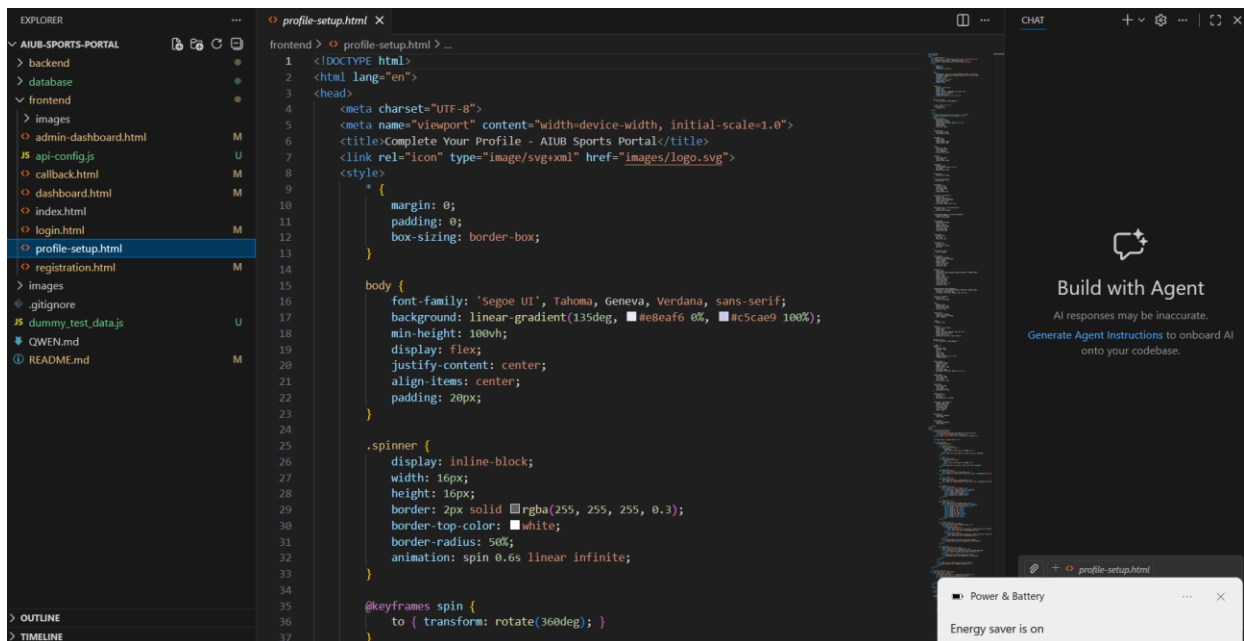
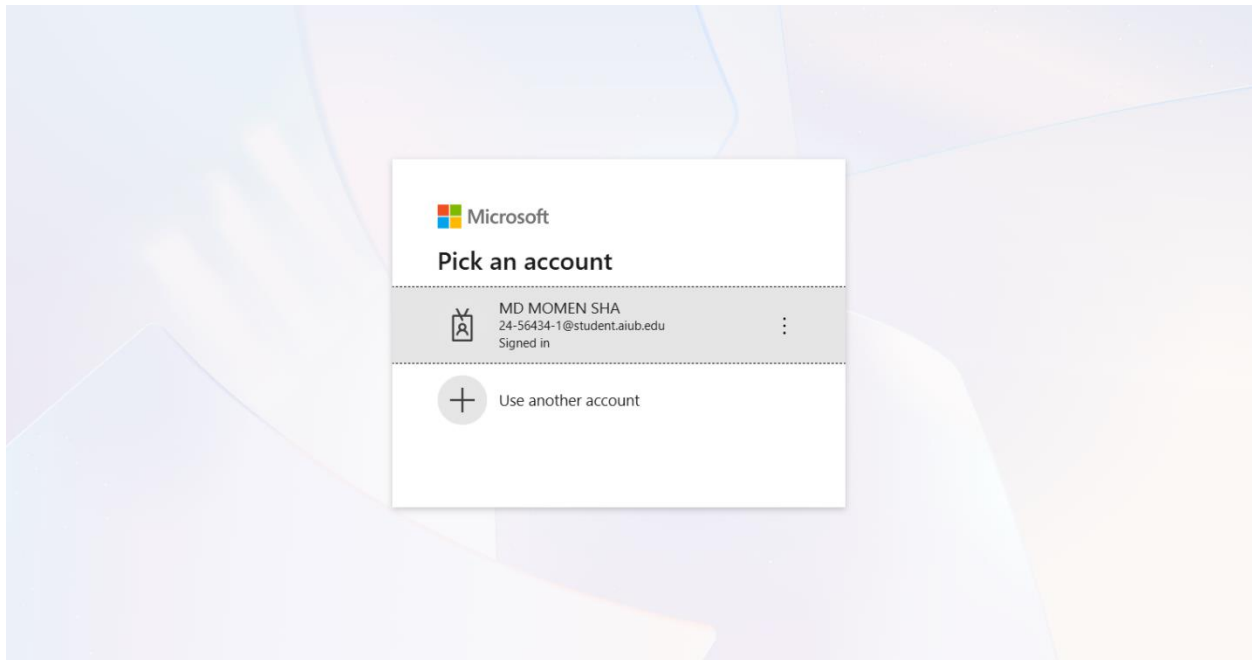
Answer: $\Pi_{full_name}(\sigma_{name_edit_count > 1}(USERS))$

Question 5: Find the full names and departments of all users.

Answer: $\Pi_{full_name, department}(USERS)$

User Interface and Code Screenshots :






AIUB
SPORTS PORTAL

Registration

Welcome MD Momen Sha

Dashboard

Registration

Featured

Messages

Bug Report

Dear Students & Faculty members, Welcome to the new AIUB Sports Portal. This platform is designed to make your sports activities easier to access, manage, and enjoy. Stay updated, stay connected, and be a part of AIUB's growing sports community.

Edit Profile

Player Name MD Momen Sha	Program Undergraduate
Player ID 24-56434-1	Department CSE
Player Email 24-56434-1@student.aiub.edu	Number 01616807177
Gender Male	Blood Group A+

No tournaments available for registration at the moment.

Registration Closed

EXPLORER

AIUB-SPORTS-PORTAL

> backend
 > database
 > frontend
 > images
 > admin-dashboard.html
 JS api-config.js
 callback.html
 dashboard.html
 index.html
 login.html
 profile-setup.html
 registration.html
 > images
 .gitignore
 JS dummy_test_data.js
 QWEN.md
 README.md

dashboard.html M


```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4
5 <meta charset="UTF-8">
6 <meta name="viewport" content="width=device-width, initial-scale=1.0">
7 <title>AIUB Sports Portal - Dashboard</title>
8 <link rel="icon" type="image/svg+xml" href="images/logo.svg">
9 <style>
10 {
11     margin: 0;
12     padding: 0;
13     box-sizing: border-box;
14 }
15
16 body {
17     font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
18     background-color: #f9fafb;
19     min-height: 100vh;
20 }
21
22 /* Header */
23 .header {
24     background: white;
25     border: 2px solid #d1d5db;
26     border-radius: 8px;
27     margin: 16px;
28     margin-bottom: 8px;
29     padding: 12px 24px;
30     display: flex;
31     align-items: center;
32     justify-content: space-between;
33 }
34
35 .header-left {
36     display: flex;
37     align-items: center;
38     gap: 16px;
      
```

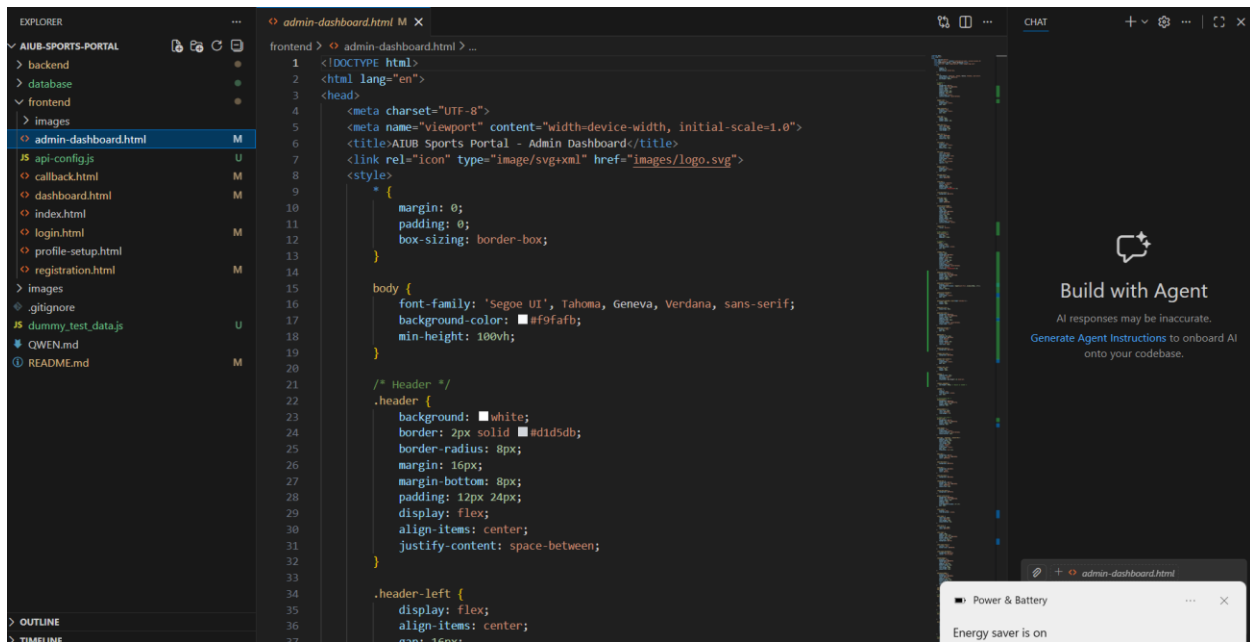
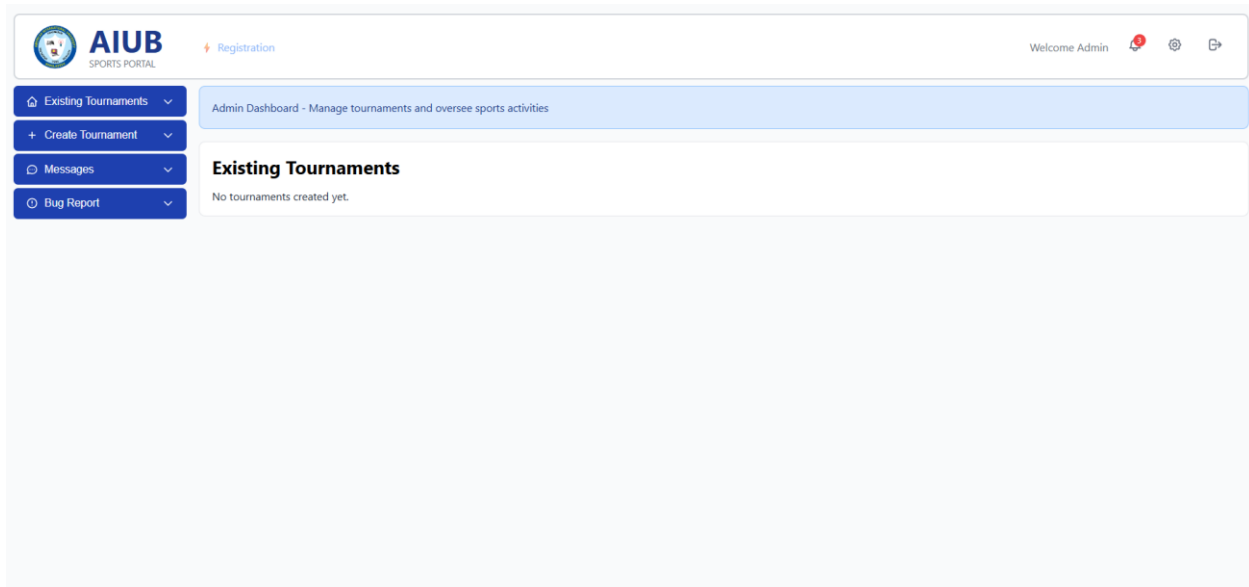
CHAT

Build with Agent

AI responses may be inaccurate.
 Generate Agent Instructions to onboard AI onto your codebase.

Power & Battery

Energy saver is on





AIUB
SPORTS PORTAL

Registration

Welcome Admin

Existing Tournaments

+ Create Tournament

Messages

Bug Report

Admin Dashboard - Manage tournaments and oversee sports activities

Create New Tournament

Tournament Title

Enter tournament title

Registration Deadline

mm/dd/yyyy --:-- --

Tournament Photo (Optional)

Choose File No file chosen

Tournament Description (Optional)

Enter tournament description...

Games

No games added yet.

+ Add game

Cancel

Create Tournament

EXPLORER

AIUB-SPORTS-PORTAL

backend

database

frontend

images

admin-dashboard.html

api-config.js

callback.html

dashboard.html

index.html

login.html

profile-setup.html

registration.html

images

gitignore

dummy_test_data.js

QWEN.md

README.md

OUTLINE

TIMELINE

registration.html M X

frontend > registration.html > ...

1 <!DOCTYPE html>

2 <html lang="en">

3 <head>

4 <meta charset="UTF-8">

5 <meta name="viewport" content="width=device-width, initial-scale=1.0">

6 <title>Tournament Registration</title>

7 <link rel="icon" type="image/svg+xml" href="images/logo.svg">

8 <style>

9 * { margin: 0; padding: 0; box-sizing: border-box; }

10

11 body {

12 font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

13 background: #f9fafb;

14 }

15

16 .header {

17 background: #ffffff;

18 border: 2px solid #ad15db;

19 border-radius: 8px;

20 margin: 16px;

21 padding: 12px 24px;

22 display: flex;

23 align-items: center;

24 justify-content: space-between;

25 }

26

27 .header-left {

28 display: flex;

29 align-items: center;

30 gap: 16px;

31 }

32

33 .logo-img {

34 width: 56px;

35 height: 56px;

36 border-radius: 50%;

37 }

CHAT

Build with Agent

AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

Power & Battery

Energy saver is on

Admin Dashboard - Manage tournaments and oversee sports activities

Create New Tournament

Tournament Title

Tournament Photo (Optional)

No file chosen

Tournament Description (Optional)

Games

No games added yet.

×

Add game to this tournament

Game

Select from default list or add a custom game.

Available for

☒ Men
 ☐ Women

By default, Men and Women are both allowed for all standard games.

Tournament format

☐ Solo
 ☐ Duo
 ☐ Duo (Mixed)
 ☐ Squad / Team (NVN)

Logic: Chess = only Solo, Pool = Solo+Duo+Mixed, Ludo = Duo+Mixed, Football = Squad only, etc.

Entry fee per person (t)

Default is 100 taka per person. Change if needed.

EXPLORER

AIUB-SPORTS-PORTAL

- back
- database
- frontend
 - images
 - admin-dash-board.html
 - api-config.js
 - callback.html
 - dashboard.html
 - index.html
 - login.html
 - profile-setup.html
 - registration.html
 - images
 - gitignore
 - dummy_test_data.js
 - QWEN.md
 - README.md

frontend > dashboard.html

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>AIUB Sports Portal - Dashboard</title>
7   <link rel="icon" type="image/svg+xml" href="images/logo.svg">
8   <style>
9     * {
10       margin: 0;
11       padding: 0;
12       box-sizing: border-box;
13     }
14     body {
15       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
16       background-color: #f9fafb;
17       min-height: 100vh;
18     }
19     /* Header */
20     .header {
21       background: white;
22       border: 2px solid #d1d5db;
23       border-radius: 8px;
24       margin: 16px;
25       margin-bottom: 8px;
26       padding: 12px 24px;
27       display: flex;
28       align-items: center;
29       justify-content: space-between;
30     }
31     .header-left {
32       display: flex;
33       align-items: center;
34       gap: 16px;
35     }
36
37

```

CHAT

Build with Agent

AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

Games

Badminton (Male)
Type: Solo • Fee: 100 BDT

Delete

Badminton (Female)
Type: Solo • Fee: 100 BDT

Delete

Badminton (Male)
Type: Duo • Fee: 100 BDT

Delete

Badminton (Female)
Type: Duo • Fee: 100 BDT

Delete

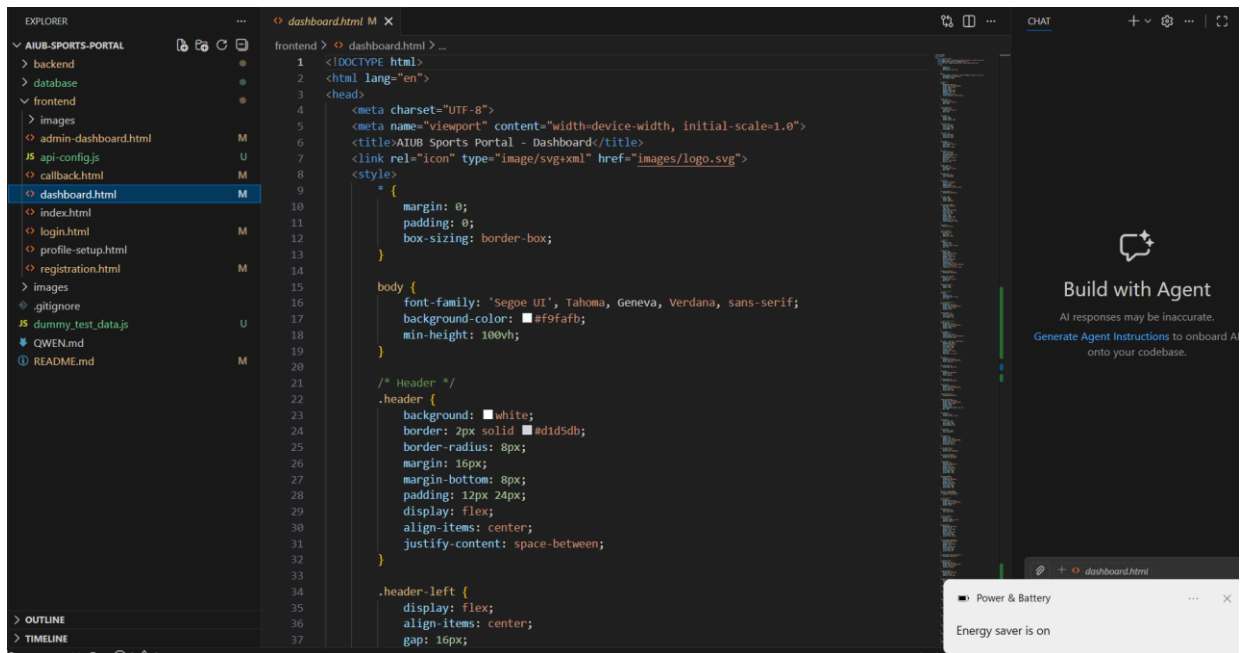
Badminton (Mix)
Type: Duo (Mixed) • Fee: 100 BDT

Delete

+ Add game

Cancel

Create Tournament



Games

Showing 5 existing games. Add more or update below.

Male Category Games

Badminton (Duo)	100 BDT	x
Badminton (Solo)	100 BDT	x

Female Category Games

Badminton (Duo)	100 BDT	x
Badminton (Solo)	100 BDT	x

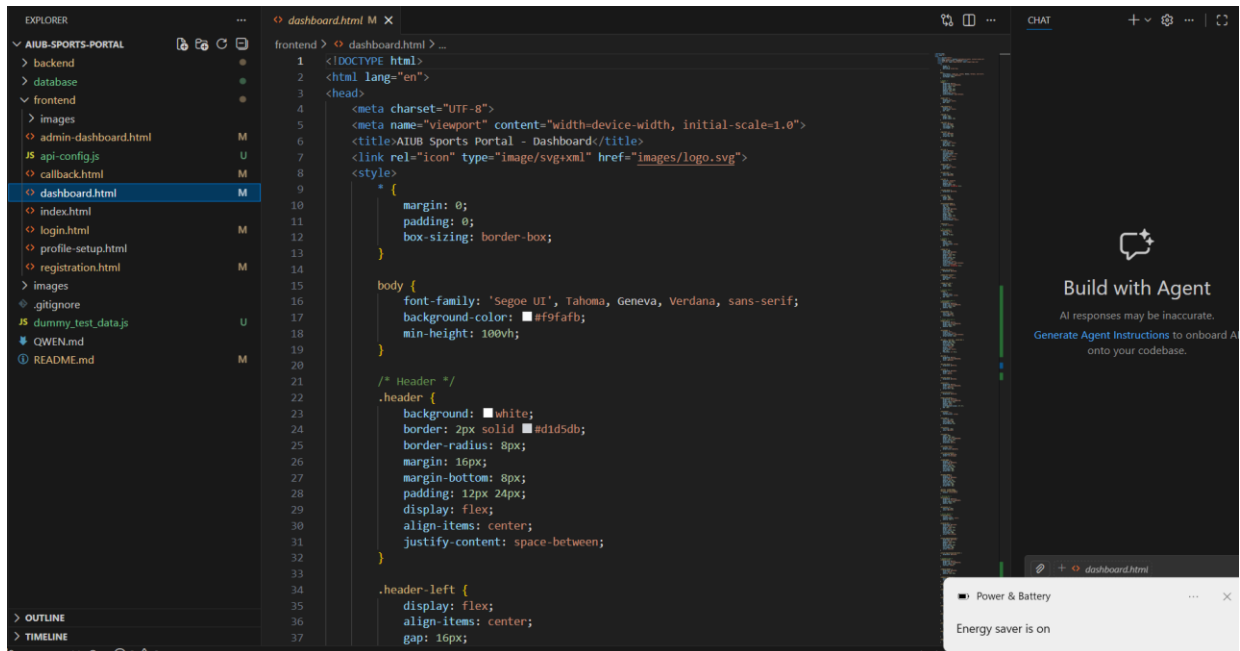
Mix Category Games

Badminton (Custom)	100 BDT	x
--------------------	---------	---

+ Add game

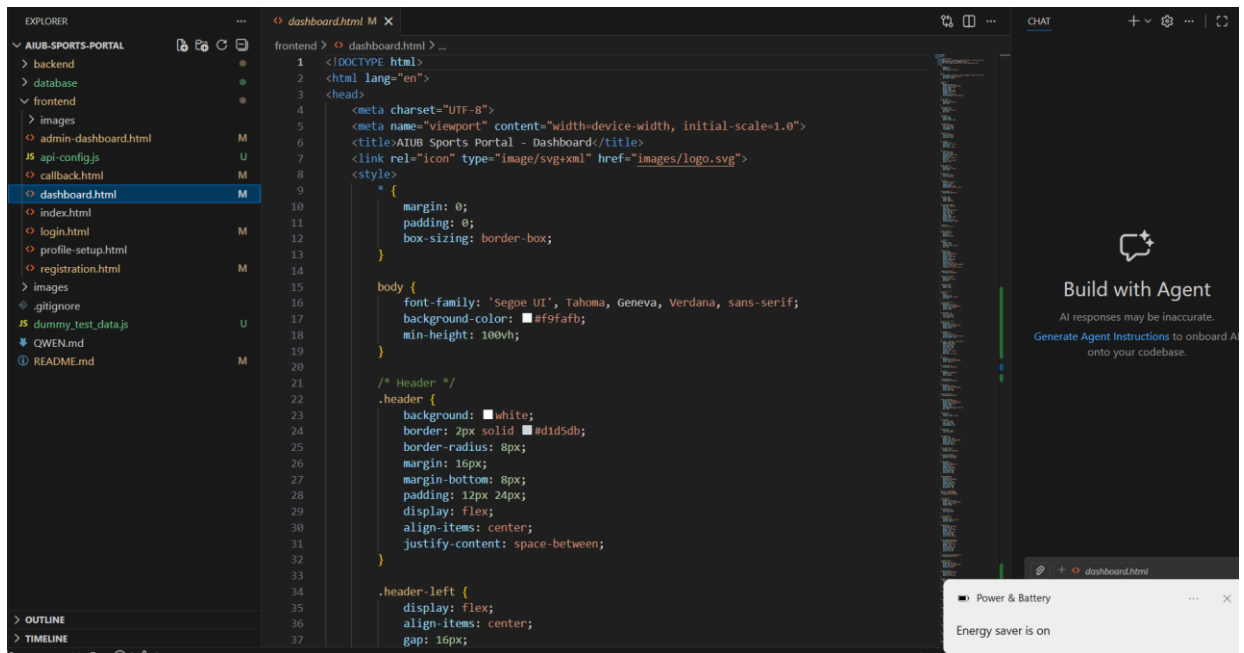
Cancel

Update Tournament



No Active Tournaments

Check back later for upcoming events!



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>AIUB Sports Portal - Dashboard</title>
7   <link rel="icon" type="image/svg+xml" href="images/logo.svg">
8 </head>
9 <body>
10   * {
11     margin: 0;
12     padding: 0;
13     box-sizing: border-box;
14   }
15   body {
16     font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
17     background-color: #f9fafb;
18     min-height: 100vh;
19   }
20
21   /* Header */
22   .header {
23     background: white;
24     border: 2px solid #d1d5db;
25     border-radius: 8px;
26     margin: 16px;
27     margin-bottom: 8px;
28     padding: 12px 24px;
29     display: flex;
30     align-items: center;
31     justify-content: space-between;
32   }
33
34   .header-left {
35     display: flex;
36     align-items: center;
37     gap: 16px;
```